Packed Pre-Constructed PVSS for Randomness Generation and E-Voting

Karim Baghery[1], Eleftheria Makri[2], and Dheeraj Kumar Suryakari[2]

[1] COSIC, KU Leuven, Leuven, Belgium
[2] LIACS, Leiden University, Leiden, Netherlands
karim.baghery@kuleuven.be, e.makri@liacs.leidenuniv.nl,
d.k.suryakari@liacs.leidenuniv.nl
May 31, 2026

Abstract. Pre-constructed Publicly Verifiable Secret Sharing (PPVSS) extends conventional Publicly Verifiable Secret Sharing (PVSS) by requiring the dealer to publish a commitment or encryption of the shared secret, enabling more efficient and versatile constructions for a variety of cryptographic protocols. In this paper, we further enhance this paradigm by introducing Packed Pre-constructed PVSS (3PVSS), which allows a dealer to encode multiple secrets within a single polynomial while maintaining the pre-constructability property. We present two constructions of 3PVSS schemes. In the first construction, the dealer publishes a single commitment to all shared secrets, providing a compact representation that is particularly suitable for applications requiring efficient communication. In the second construction, the dealer publishes individual commitments to each secret, enabling greater flexibility in applications that require independent verification of multiple shared values. Both schemes preserve the key advantages of PPVSS, including the optimistic reconstruction approach enabled by pre-constructability. We demonstrate that these two variants naturally support different applications. Using the first 3PVSS construction, we revisit the ALBATROSS randomness generation protocol proposed by Cascudo and David (ASIACRYPT 2020) and present a more efficient variant with reduced computation and communication costs. As an application of the second construction, we extend the universally verifiable e-voting protocol recently proposed by Baghery, Knapen, Nicolas, and Rahimi (ACNS 2025) to support multi-candidate elections, while preserving universal verifiability. Our results show that the resulting protocols outperform their original counterparts in terms of efficiency.

Keywords: Publicly Verifiable Secret Sharing · Pre-Constructed PVSS · E-voting · Randomness Generation · Packed Secret Sharing.

1 Introduction

Secret sharing is a fundamental cryptographic technique that allows an entity, called the *dealer* to securely distribute a secret s among a set of n parties, such

that only authorized subsets of these n parties can reconstruct the secret. Unauthorized subsets of the parties learn nothing about the secret. Shamir secret sharing [24] is one of the first secret sharing schemes (appeared concurrently with Blakley's scheme [6] in 1979), and one of the most widely used constructions. Secret sharing serves as a core building block in numerous cryptographic protocols designed to eliminate reliance on a single trusted entity, including applications in multiparty computation and threshold cryptography.

Classical secret sharing schemes (e.g., Shamir secret sharing [24]) on their own offer passive security. This means that privacy is guaranteed against an adversary who passively corrupts any subset of parties smaller than the authorized subsets. In this context, passively refers to trying to learn as much information as possible about the secret, without deviating from the protocol specification. As such, attacks against the integrity of the shares are not considered. A malicious dealer/party is allowed to deviate from the protocol specification in an arbitrary manner, for instance by distributing/providing incorrect values during the secret sharing/reconstruction phase. Such adversaries can be thwarted by *Verifiable* Secret Sharing (VSS) schemes [1, 3, 4, 7, 13, 15, 22, 25]. VSS allows the parties to verify the consistency of the shares provided by the dealer/parties during the sharing/reconstruction phase, and if a corrupted dealer/party attempts to provide inconsistent shares, this behavior is detected. *Publicly* Verifiable Secret Sharing (PVSS) [4, 8–10, 18–21, 23] goes a step further than VSS, allowing anyone, including external observers who do not participate in the protocol, to verify that the shared values are consistent. This is particularly useful in application scenarios where public auditability and minimal interaction are necessary, such as e-voting systems, auctions, privacy-preserving finance, distributed randomness generation, and privacy-preserving federated learning.

The notion of *Pre-constructed* PVSS, where the dealer is required to additionally commit to the shared (reconstructed) secret, was recently introduced by Baghery, Knapen, Nicolas, and Rahimi [5]. As observed in [5], this notion not only improves the efficiency of existing PVSS-based applications, but also offers greater versatility than standard PVSS. Baghery et al. [5] further show that the PVSS by Schoenmakers [23], as well as its pairing-based variant by Heidarvand and Villar [20] are a special case of Pre-constructed PVSS. However, their cost, notably the quadratic in the number of parties cost of Schoenmakers' PPVSS, in terms of exponentiations required during verification, gives rise to new, more efficient schemes. Several follow-up works [4, 8, 9, 11] have improved the efficiency of the original PVSS by Schoenmakers [23], albeit at the cost of losing the *re-constructability* property.

Packed secret sharing schemes [15, 17] improve the efficiency of Shamir secret sharing by encoding ℓ secrets into one single polynomial. This can be achieved, without increasing the share size. However, given that for a t-degree polynomial at most $t+1$ secrets can be securely encoded, to achieve packing of ℓ secrets the degree of the polynomial should be increased to $t+\ell$ and the corruption threshold should be decreased such that $n \geq 2t + \ell$ in the synchronous communication model. As a result, an inherent tradeoff between privacy and robustness arises in

packed schemes, concretely offering t-privacy and $(t+\ell)$-reconstruction. Building on this idea, *packed* variants of VSS and PVSS [2, 9] extend their standard counterparts to the packed setting. In these schemes, a dealer can distribute multiple secrets simultaneously while allowing parties, or even the public in the case of PVSS, to verify the correctness of the distributed or broadcasted shares. This significantly improves the amortized efficiency of PVSS and VSS protocols when multiple secrets need to be shared.

Our Contribution. Our contributions are summarized as follows.

Packed Pre-Constructed PVSS (3PVSS). We first extend the notion of Pre-Constructed PVSS introduced by Baghery et al. [5] to the packed setting, resulting in a new primitive that we call *Packed Pre-constructed PVSS* (3PVSS). We then present two practical 3PVSS constructions based on the PPVSS scheme Λ_{RO} of Baghery et al. [5], which is built in the Random Oracle (RO) model. Both constructions allow a dealer to share ℓ secrets simultaneously in a publicly verifiable manner while also publishing well-formed commitments to the ℓ shared (pre-constructed) secrets. In the first scheme, denoted Λ_{RO}^{indv}, the dealer publishes ℓ individual commitments corresponding to the ℓ shared (pre-constructed) secrets and proves the well-formedness of these commitments together with the encrypted shares. In the second scheme, denoted Λ_{RO}^{single}, the dealer publishes a *single* commitment to all ℓ secrets and proves its well-formedness along with the encrypted shares. Naturally, Λ_{RO}^{single} achieves lower communication complexity since only a single commitment is broadcast for all secrets. However, we show that both variants offer different advantages and may be preferable in different applications. While our schemes are presented in the RO model, the techniques we develop appear sufficiently general to extend to pairing-based PPVSS schemes [5] constructed in the plain model.

A Universally Verifiable Multi-Candidate E-Voting Protocol. Schoenmakers [23] showed how his (P)PVSS scheme can be used to build a universally verifiable e-voting protocol for binary elections without requiring a trusted third party. However, in his protocol the verification cost per voter is $O(nt)$, where (n, t) are the parameters in the (P)PVSS scheme. More recently, Baghery et al. [5] used their PPVSS scheme Λ_{RO} to reduce this verification cost to $O(n)$. As our third contribution, we use our 3PVSS scheme Λ_{RO}^{indv} to design a new variant of the e-voting protocol of Baghery et al. [5] that supports multi-candidate elections. We show that the resulting protocol not only supports elections with multiple candidates but also improves efficiency compared to the original scheme.

More Efficient ALBATROSS Random Beacon. ALBATROSS [9] is an efficient distributed randomness generation protocol that uses an ℓ-packed PVSS scheme to generate ℓ^2 random values. As our second contribution, we revisit the ALBATROSS protocol by replacing its underlying packed PVSS with our proposed 3PVSS construction Λ_{RO}^{single}. This results in a new variant of the protocol that achieves improved performance in terms of both computational and communication costs.

2 Preliminaries

Notation. We let λ denote a security parameter. A function is called *negligible in* X, written $\mathsf{negl}(X)$, if for any constant c, there exists some X_0, such that $f(X) < X^{-c}$ for $X > X_0$. A function that is negligible in the security parameter λ is simply called negligible. We use the assignment operator $\leftarrow$ to denote uniform sampling from a set Ξ, e.g. $x \leftarrow \Xi$. We write $\mathbb{Z}_N := \mathbb{Z}/N\mathbb{Z}$ and $\mathbb{Z}_N[X]_t$ for polynomials of degree t in the variable X and with coefficients in $\mathbb{Z}_N$. For $n \in \mathbb{N}$, we write $[n] = \{1, \ldots, n\}$. Finally all logarithms are in base 2.

2.1 Packed Shamir Secret Sharing

(n, t, ℓ)-Packed Shamir secret sharing ([17], [24]) scheme is a generalization of (n, t)-Shamir's secret sharing scheme [24] where a dealer embeds multiple secrets in a single polynomial. Concretely, if the dealer wants to secret share ℓ secrets, they will embed those secrets in a single $t + \ell - 1$ degree secret polynomial with coefficients in a field, which is secret shared amongst n parties such that any $t+\ell$ parties can reconstruct back the polynomial and thereby obtain the secrets. The scheme is summarized in the Figure 1.

Packed Shamir Secret Sharing

Given ℓ secrets to share amongst n parties, where at most t of them can be *(passively)* corrupt, the (n, t, ℓ)-Packed Shamir secret sharing scheme description is as follows:

Sharing Algorithm:

- Dealer constructs the secret polynomial $f \in \mathbb{Z}_q[X]_{t+l-1}$ via the lagrange interpolation by choosing $t + \ell$ elements in $\mathbb{Z}_q$ where ℓ of them are secrets, $\{s_i\}_{i=0}^{\ell-1}$, with $f(-i) = s_i$ for all i and remaining t are chosen uniformly at random in $\mathbb{Z}_q$.
- Each party P_i receives their share $f(i)$ from the Dealer for each $i \in \{1, \ldots, n\}$

Reconstruction Algorithm:

- Any $\mathcal{Q}$ set containing at least $t+\ell$ parties can use the lagrange interpolation to compute $\{s_i\}_{i=0}^{\ell-1}$ as follows:

$$s_m = \sum_{i \in \mathcal{Q}} f(i) \left[\prod_{j \in \mathcal{Q}, j \neq i} \frac{-m-j}{i-j} \right] \qquad , m \in \{0, \ldots, \ell-1\}$$

- The secrets $\{s_i\}_{i=0}^{\ell-1}$ are outputted as the result.

Fig. 1: Packed Shamir Secret Sharing

Prover: Given the statement $(g, h, a, b) \in \mathbb{G}$ and the witness value $x \in \mathbb{Z}_q$, proceed as follows and output a proof π.

1. Sample $r \leftarrow\$ \mathbb{Z}_q$ uniformly at random; and set $c_1 = g^r$ and $c_2 = h^r$.
2. Set $d \leftarrow \mathcal{H}(a, b, c_1, c_2)$, where $\mathcal{H}$ is a random oracle.
3. Set $z = r + d \cdot x \mod q$; and Return $\pi := (d, z)$

Verifier: Given the statement $(g, h, a, b) \in \mathbb{G}$ and the proof $\pi = (d, z)$, checks if $d = \mathcal{H}(a, b, \frac{g^z}{a^d}, \frac{h^z}{b^d})$ and outputs `true` or `false`.

Fig. 2: Chaum-Pedersen NIZK proof of knowledge for DLEQ [12].

Though packed Shamir secret sharing is useful over naive approach to secret share multiple secrets, it comes with a cost of sacrificing the corruption tolerance of the scheme. More precisely, it requires $n \geq 2t + \ell$, i.e., if one were to secret share more secrets for a fixed n number of shareholders then the corruption tolerance t of the scheme will decrease.

2.2 NIZK Proof for DL Equality

Let $\mathbb{G}$ be a group with hard DL, and g, h be two group elements, where g is the group generator. Let a prover aim to convince a verifier that for the public statement g, h, a, b, he knows a witness x which holds in the following relation,

$$R_{DLEQ} = \{(g, h, a, b), x \mid a = g^x \wedge b = h^x\}. \tag{1}$$

This relation is known as DL EQuality (DLEQ). In [12], Chaum and Pedersen introduced an efficient NIZK proof of knowledge for DLEQ, as summarized in Fig. 2. This protocol is widely use in various cryptographic protocols, including threshold decryption, e-voting systems, PVSS and PPVSS schemes, etc.

2.3 NIZK Proof of Binary Vote

In the ballot casting phase of the e-voting protocol proposed by Schoenmakers [23], a voter V casts a binary vote $v \in \{0, 1\}$ by running the sharing phase for the (P)PVSS scheme, using a random secret $s \in \mathbb{Z}_q$, and computing the value $U = G^{s+v}$. In addition, the voter generates a proof π_U to show that $v \in \{0, 1\}$ without revealing any information on v. The proof π_U refers to the commitment $C_0 = g^s$, which is published as part of the (P)PVSS transcript, and proves that:

$$\log_G U = \log_g C_0 \ \vee \ \log_G U = 1 + \log_g C_0.$$

The proof π_U is constructed using the technique of [14], and its protocol can be summarized as follows:

1. The prover sets $a_v = g^w$ and $b_v = G^w$ for random $w \in_R \mathbb{Z}_q$. The prover also sets $a_{1-v} = g^{r_{1-v}} C_0^{d_{1-v}}$ and $b_{1-v} = G^{r_{1-v}} (U/G^{1-v})^{d_{1-v}}$, for random $r_{1-v}, d_{1-v} \in_R \mathbb{Z}_q$. The prover sends a_0, b_0, a_1, b_1 in this order to the verifier.

2. The verifier sends a random challenge $c \in_R \mathbb{Z}_q$ to the prover.
3. The prover sets $d_v = c - d_{1-v} \pmod q$ and $r_v = w - sd_v \pmod q$, and sends d_0, r_0, d_1, r_1 in this order to the verifier.
4. The verifier checks that $c = d_0 + d_1 \pmod q$ and that

$$a_0 = g^{r_0} C_0^{d_0}, \quad b_0 = G^{r_0} U^{d_0}, \quad a_1 = g^{r_1} C_0^{d_1}, \quad \text{and} \quad b_1 = G^{r_1} \left(\frac{U}{G}\right)^{d_1}.$$

The protocol is proven to satisfy completeness, special soundness and honest verifier zero-knowledge; hence, its non-interactive version achieves ZK and releases no information on v in the random oracle model.

2.4 NIZK Proof for PDL Problem

Polynomial Discrete Problem. In [4], Baghery generalize Discrete Logarithm (DL) relation over polynomials and introduce the Polynomial Discrete Logarithm (PDL) relation denoted as R_{PDL}, which is defined as follows,

$$R_{PDL} = \{(g, x_i, F_i), f(X) \mid F_i = g^{f(x_i)} \text{ for } i = 1, 2, \ldots, n\}.$$

Here, $f(X) \in \mathbb{Z}_q[X]_t$ is a (at most) degree $t \leq n-1$ witness polynomial with coefficients from $\mathbb{Z}_q$, and $\{x_i\}_{i=1}^n$ are n *distinct* elements from $\mathbb{Z}_q$. The R_{PDL} relation is based on the PDL problem, defined as follows, which is shown to be reducible to the DL problem [4].

Definition 1 (Polynomial Discrete Logarithm Problem [4]). *Let $\mathbb{G}$ be a finite cyclic group of order q generated by g. Given $F_1, \ldots, F_n$ from $\mathbb{G}$ and distinct elements $x_1, \ldots, x_n$ from $\mathbb{Z}_q$, find a polynomial $f(X) \in \mathbb{Z}_q[X]_t$ of (at most) degree t, where $0 \leq t \leq n-1$, such that $F_i = g^{f(x_i)}$ for all $i = 1, \ldots, n$.*

In other words, an algorithm $\mathcal{A}$ has advantage ϵ in solving PDL in $\mathbb{G}$ if

$$\Pr[\mathcal{A}(x_1, \ldots, x_n, g, g^{f(x_1)}, \ldots, g^{f(x_n)}) = f(X)] \geq \epsilon$$

where $f(X) \in \mathbb{Z}_q[X]_t$ is (at most) a degree-t polynomial with $0 \leq t \leq n-1$, and the probability is over the random choice of generator $g \in \mathbb{G}^*$ and the distinct choice of $x_1, \ldots, x_n$ in $\mathbb{Z}_q$.

Baghery [4] also presented a Non-Interactive Zero-Knowledge (NIZK) Proof-of-Knowledge (PoK) scheme π_{PDL}, that allows a prover to prove knowledge of a witness for R_{PDL} relation.

π_{PDL}

Let g be a group generator sampled in the setup phase, $[(g, x_1, \ldots, x_n, F_1, \ldots, F_n), f(X)] \in R_{PDL}$ and $\mathcal{H}$ be a random oracle (RO).

Prover

- Samples $r \in_R \mathbb{Z}_q[X]_t$ uniformly at random and sets $\Gamma_i = g^{r(x_i)}$ for $1 \leq i \leq n$.
- Sets $d \leftarrow \mathcal{H}(F_1, \ldots, F_n, \Gamma_1, \ldots, \Gamma_n)$, where $\mathcal{H}$ is an agreed upon Random Oracle (RO).
- Sets $z(X) \equiv r(X) + df(X) \pmod q$ and returns the proof(/transcript) $\pi := (d, z(X))$.

Verifier

- First checks if z is at most a t degree polynomial in $\mathbb{Z}_q[X]_t$. If so, they proceed with the next step.
- Checks if $d \leftarrow \mathcal{H}(F_1, \ldots, F_n, \frac{g^{z(x_1)}}{F_1^d}, \ldots, \frac{g^{z(n)}}{F_n^d})$.
- If first two steps are correct then they output **true**, otherwise **false**.

Fig. 3: A NIZK PoK for Polynomial DL based on Schoenmakers' PVSS

3 Packed Pre-Constructed PVSS

For Guaranteed Output Delivery (GOD), several distributed applications rely on techniques based on VSS. For example, the randomness beacon ALBATROSS [9] employs PVSS schemes. These secret sharing schemes allow a dealer to prove the validity of encrypted shares to anyone using ZK proofs, without revealing any information about the secret or the shares themselves. However, when the dealer later reveals the secret, verifying its correctness typically requires simulating the secret-sharing phase of the (P)VSS protocol, which is inefficient. This inefficiency arises because the dealer initially publishes only encrypted shares, and the accompanying ZK proof guarantees consistency of the shares with *some* secret, without revealing any information about the secret itself. In 2025, Pre-Constructed PVSS (PPVSS) was introduced in [5]. This primitive extends PVSS with additional functionality and provides a novel and significantly more efficient approach for secret reconstruction when the dealer is available. This setting is common in distributed protocols built on VSS, such as DKG protocols and randomness beacons. In essence, the new reconstruction approach enables much more efficient verification of the revealed secret.

The goal of this section is to extend the notion of PPVSS to the packed setting, which we refer to as *Packed Pre-constructed PVSS* (3PVSS). Conceptually, 3PVSS can be viewed as a PPVSS construction built on top of *packed* secret sharing schemes. While packed secret sharing is known to sacrifice some robustness, it has been shown in [9] that packing can nevertheless be leveraged to design more efficient randomness beacon protocols. In this paper, we show that

3PVSS not only improves the efficiency of such applications in many settings, but can also enable new applications.

3.1 Definitions

We begin by naturally extending the definition of PPVSS from [5] to the packed setting. In this variant, the dealer publishes commitments to the shared secrets and publicly proves the well-formedness of the commitment(s) together with the encrypted shares. At the level of the definition, we do not specify the particular commitment mechanism used by the dealer. Later in this section, we present two concrete 3PVSS constructions: in the first, the dealer commits to all packed secrets using a single group element, while in the second the dealer publishes individual commitments for each secret.

Definition 2 (3PVSS). *A 3PVSS scheme should have four algorithms, namely, Initial, Share, Verify and Reconstruction whose descriptions are as follows:*

- ***Initial*** $(1^\lambda) \rightarrow (\{PK_i, SK_i\}_{i=1}^n, h_0)$*: When given* 1^λ*, each party* P_i *for* $1 \leq i \leq n$ *registers their public key* PK_i *in a public ledger and withholds the corresponding secret key* SK_i*. Also, all parties and the dealer* D *agree on a vector* $\vec{h}_0$ *of commitment keys to commit to* ℓ *secrets.*
- ***Share*** $(n, t, \vec{s}, \{PK_i\}_{i=1}^n, \vec{h}_0) \rightarrow (\vec{y}_0, \{y_i\}_{i=1}^n, \pi_{3PVSS})$*: It secret shares* $\vec{s}$*, a vector of secrets of length* ℓ*, to obtain their shares* $\{f_i\}_{i=1}^n$*. For* $1 \leq i \leq n$*, uses the public key* PK_i *to encrypt* f_i *and obtain the encrypted share* y_i*. Now, it uses the commitment keys in* $\vec{h}_0$ *to commit to the secrets in* $\vec{s}$ *to obtain a vector of committed secrets,* $\vec{y}_0$*. Then, it generates a NIZK proof* π_{3PVSS} *to prove that* $\vec{y}_0$ *consists valid secrets' commitments and* $\{y_i\}_{i=1}^n$ *has valid encryptions of their corresponding shares. Finally, it returns* $(\vec{y}_0, \{y_i\}_{i=1}^n, \pi_{3PVSS})$*.*
- ***Verify*** $(n, t, \vec{h}_0, \vec{y}_0, \{PK_i, y_i\}_{i=1}^n, \pi_{3PVSS}) \rightarrow$ *true/false: This algorithm checks if* π_{3PVSS} *is valid for* $\{\vec{y}_0, y_1 \ldots, y_n\}$ *and then returns true, otherwise false.*
- ***Reconstruct****: There are two approaches as follows:*
  - ***(Optimistic)*** *Reconstruction*$^{opt}(\{\vec{h}_0, \vec{s}, \vec{y}_0\}) \rightarrow (\vec{s}$ *or* $false)$*: Given the inputs* $(\vec{h}_0, \vec{s}, \vec{y}_0)$ *a party checks if* $\vec{y}_0$ *is a valid commitment of* $\vec{s}$*. If so, it returns* $\vec{s}$*; otherwise it returns* $false$*.*
  - ***(Pessimistic)*** *Reconstruction*$^{pes}(\{y_i, SK_i\}_{i \in \mathcal{Q}, |\mathcal{Q}|=t+\ell}] \rightarrow (\vec{s}$ *or* $false)$*: Given any* $t+\ell$ *encrypted shares along with corresponding secret keys, it outputs the secret* $\vec{s}$ *or* $false$*. This can be done in two phases as follows:*
    - ***Decryption of the shares****, each party* $P_i \in \mathcal{Q}$ *decrypts* y_i *to obtain* f_i *using its* SK_i*. Then it generates a NIZK proof* π_i^{Dec} *which proves that* f_i *is the correct decryption of* y_i*. Now,* P_i *publishes* (f_i, π_i^{Dec})*.*
    - ***Share pooling****, a verifier (not necessarily from the shareholders) checks if proof* π_i^{Dec} *is correct for each* $P_i \in \mathcal{Q}$*. If any check fails, then it returns* $false$*; otherwise it applies a reconstruction procedure to the set of valid shares,* $\{f_i\}_{i \in \mathcal{Q}, |\mathcal{Q}|=t+\ell}$*, and returns secret(s)* $\vec{s}$*.*

A 3PVSS is said to be secure:

- **Correctness**: If the parties follow the protocol, then the *Verify* algorithm returns *true* and the *Reconstruct* algorithm returns $\vec{s}$ irrespective of which approach. For any integer $n \geq t+\ell$ with $t > 0$, a 3PVSS is said to be correct for $\left(\{PK_i, SK_i\}_{i=1}^n, \vec{h}_0\right) \leftarrow Initial(1^\lambda)$ when it satisfies the following

$$Pr\left[\begin{array}{c}(\vec{y}_0, \{y_i\}_{i=1}^n, \pi_{3PVSS}) \leftarrow Share\big(n, t, \vec{s}, \{PK_i\}_{i=1}^n, h_0\big) : \\ true \leftarrow Verify(n, t, \vec{h}_0, \vec{y}_0, \{PK_i, y_i\}_{i=1}^n, \pi_{3PVSS})\end{array}\right] = 1,$$

$$Pr\left[\begin{array}{c}(\vec{y}_0, \{y_i\}_{i=0}^n, \pi_{3PVSS}) \leftarrow Share\big(n, t, \vec{s}, \{PK_i\}_{i=1}^n, \vec{h}_0\big), \\ \vec{s}' \leftarrow Reconstruction^{opt}(\vec{h}_0, \vec{s}, \vec{y}_0) \bigvee \\ \vec{s}' \leftarrow Reconstruction^{pes}(\{y_i, SK_i\}_{i\in\mathcal{Q}, |\mathcal{Q}|=t+\ell}) : \vec{s}' = \vec{s}\end{array}\right] = 1.$$

- **Verifiability**: If *Verify* returns *true*, then *(Optimistic) Reconstruct*^{opt} and/or *(Pessimistic) Reconstruct*^{pes} output being $\vec{s}$ is the actual secret vector of whom the shares are encrypted. Moreover, with high probability, $\{y_i\}_{i=1}^n$ are valid encryptions of the shares of same secrets in the vector and $\vec{y}_0$ is a valid vector of commitment(s) of those secrets. More formally, given λ, for any integers $n \geq 2t + \ell$ and $t > 0$, a 3PVSS is said to be verifiable if for $\left(\{PK_i, SK_i\}_{i=1}^n, \vec{h}_0\right) \leftarrow Initial(1^\lambda)$ any $\mathcal{PPT}$ adversary $\mathcal{A}$, we have:

$$Pr\left[\begin{array}{c}(\vec{y}_0, \{y_i\}_{i=1}^n, \pi_{3PVSS}) \leftarrow \mathcal{A}\big(n, t, \{PK_i\}_{i=1}^n, \vec{h}_0\big), \\ true \leftarrow Verify(n, t, \vec{h}_0, \vec{y}_0, \{PK_i, y_i\}_{i=1}^n, \pi_{3PVSS}) : \\ \exists \vec{s}, \forall Q \in [n], \quad \vec{s} \leftarrow Reconstruction^{opt}(\{\vec{h}_0, \vec{s}, \vec{y}_0\}) \bigvee \\ \vec{s} \leftarrow Reconstruction^{pes}(\{y_i, SK_i\}_{i\in\mathcal{Q}})\end{array}\right] \geq 1 - negl(\lambda),$$

  where $\mathcal{Q}$ is the set of honest parties and $|\mathcal{Q}| \geq t + \ell$.
- **IND1-Secrecy (Indistinguishability of Secrets)**: Before reconstruction phase, any amount of public information along with secret keys of at most t parties will give absolutely no information about the secrets $\vec{s}$. More formally, for integers $n > 1$ and $t + \ell \leq n$, the 3PVSS is said to satisfy *IND1-Secrecy* if for any $\mathcal{PPT}$ adversary $\mathcal{A}$ corrupting at most t parties has negligible advantage in the following game played against a challenger.
  - The challenger runs $Initial(1^\lambda)$ of 3PVSS to obtain $\{PK_i, SK_i\}_{i=1}^n, \vec{h}_0$ and sends all public information along with secret information of all corrupted parties to $\mathcal{A}$.
  - The challenger chooses two vectors of secrets, $\vec{s}_0$ and $\vec{s}_1$ at random in the space of secrets. Furthermore, it chooses $b \in \{0, 1\}$ uniformly at random and runs $Share(n, t, \vec{s}_0, \{PK_i\}_{i=1}^n, \vec{h}_0)$ algorithm of the 3PVSS scheme and obtains $(\vec{y}_0, \{y_i\}_{i=1}^n, \pi_{3PVSS})$. Finally, it sends all public information generated in *Share* phase together with $\vec{s}_b$.
  - $\mathcal{A}$ guesses a bit $b' \in \{0, 1\}$.

  The advantage of $\mathcal{A}$ is defined to be $|Pr[b' = b] - \frac{1}{2}|$.

3.2 A Packed Pre-Constructed PVSS with Individual Commitments

In this section, we present our first practical 3PVSS scheme, denoted Λ_{RO}^{indv}. This construction extends the PPVSS scheme Λ_{RO} [5], which itself builds on the

Packed Pre-Constructed PVSS Scheme Λ_{RO}^{indv}

Initial: All parties $\{P_i\}_{i=1}^n$ and dealer D agree on the prime field $\mathbb{Z}_q$, a group $(\mathbb{G}, \times)$ of order q with a generator g, random oracle $\mathcal{H}$. Also, each party P_i registers their public key PK_i in the public ledger and all agree on a commitment key, $h_0 \in \mathbb{G}$.

Share: Given $(n, t, \vec{s}, \{PK_i\}_{i=1}^n, \vec{h}_0)$, D acts as follows and returns $(\vec{y}_0, \{y_i\}_{i=1}^n, \pi_{3PVSS})$:

- Dealer D samples a $t+\ell-1$-degree polynomial $f \in \mathbb{Z}_q[X]$ uniformly at random and sets $\{g^{f_j} : f_j = f(j)\}_{j=-(\ell-1)}^0$ as the set of all secrets, where $\vec{s} := (f_{-(\ell-1)}, \ldots, f_{-1}, f_0)$.
- For each $1 \leq i \leq n$, D uses PK_i and $f_i = f(i)$ and computes encrypted values $y_i = PK_i^{f_i}$. D also computes the commitments to secrets $\{g^{f_j}\}_{j=-(\ell-1)}^0$ as $\{y_j = h_0^{f_j}\}_{j=-(\ell-1)}^0$.
- D uses the PDL proof scheme [4] for $i \in (-(\ell-1), \ldots, n)$ to generate π_{3PVSS} as follows:
  - Samples a $t+\ell-1$-degree polynomial $r \in \mathbb{Z}_q[X]$ uniformly at random and computes $\{c_j = h_0^{r(j)}\}_{j=-(\ell-1)}^0$ and $c_i = PK_i^{r(i)}$ for $1 \leq i \leq n$.
  - Using the random oracle $\mathcal{H}$, $d = \mathcal{H}(y_{-(\ell-1)} \ldots, y_n, c_{-(\ell-1)}, \ldots, c_n)$ is computed.
  - Sets $z(X) = r(X) + df(X)$, hence $\pi_{3PVSS} = (d, z(X))$ is obtained.
- D broadcasts $\vec{y_0} := (y_{-(\ell-1)}, \ldots, y_0)$, encrypted shares $\{y_i\}_{i=1}^n$, and $\pi_{3PVSS} := (d, z(X))$.

Verify: Given $(n, t, \vec{h}_0, \vec{y}_0, \{PK_i, y_i\}_{i=1}^n, \pi_{3PVSS})$, the verifier acts as follows:

- Checks if $z(X)$ is a (at most) $t+\ell-1$-degree polynomial or not.
- Checks if $d = \mathcal{H}(y_{-(\ell-1)}, \ldots, y_n, \frac{h_0^{z(-(\ell-1))}}{y_{-(\ell-1)}^d}, \ldots, \frac{h_0^{z(0)}}{y_0^d}, \frac{PK_1^{z(1)}}{y_1^d}, \ldots, \frac{PK_n^{z(n)}}{y_n^d})$.
- Outputs *true* if both of the above checks are satisfied, otherwise *false*.

Reconstruct: Reconstruction of the secrets $\{g^{f_j}\}_{j=-(\ell-1)}^0$ can be done by two approaches:

- **Optimistic:** First D publishes $\{f_j\}_{j=-(\ell-1)}^0$. Then given $g, h_0, \{y_j\}_{j=-(\ell-1)}^0$ parties check if $\{y_j = h_0^{f_j}\}_{j=-(\ell-1)}^0$ and returns $\{g^{f_j}\}_{j=-(\ell-1)}^0$ if all checks pass, if not they return *false*.
- **Pessimistic:** Any qualified set $\mathcal{Q}$, where $|Q| \geq t+\ell$, of parties act as follows:
  - Each party $P_i \in \mathcal{Q}$ decrypts y_i using SK_i corresponding to PK_i and obtains g^{f_i} and then publishes it along with a DLEQ proof π_{DLEQ} to prove the correctness of decryption.
  - If $\mathcal{Q}$ consists at least $t+\ell$ honest parties, they can use Lagrange interpolation to compute the secrets $\{g^{f_j}\}_{j=-(\ell-1)}^0$ as follows:

$$g^{f_j} = \prod_{i \in \mathcal{Q}} (g^{f_i})^{\prod_{k \in \mathcal{Q}, k \neq i} \frac{-k-j}{i-j}} = g^{\sum_{i \in \mathcal{Q}} f_i \prod_{k \in \mathcal{Q}, k \neq i} \frac{-k-j}{i-j}}.$$

Fig. 4: Λ_{RO}^{indv}: An efficient 3PVSS protocol with individual commitments

PVSS schemes proposed in [4, 9]. In Λ_{RO}^{indv}, given the public keys of the parties, a vector of secrets $\vec{s}$ of length ℓ, and a commitment key $\vec{h}_0 := h_0 \in \mathbb{G}$, the dealer shares the secrets using packed Shamir secret sharing and encrypts the resulting shares under the parties' public keys. In addition, the dealer computes ℓ individual DL commitments to the ℓ shared secrets. The dealer then generates and broadcasts a NIZK proof π_{3PVSS} demonstrating the well-formedness of both the encrypted shares and the committed secrets. The full description of Λ_{RO}^{indv} is given in Figure 4, and its security is established in the following theorem.

Theorem 1. *Assuming Polynomial Discrete Logarithm (PDL) and Decisional Diffie-Hellman (DDH) are computationally hard, Λ_{RO}^{indv} is a secure 3PVSS scheme against any static computationally bounded adversary, corrupting up to t out of n parties, whenever $n \geq 2t+\ell$ with $\ell \geq 1$ being the total number of secrets.*

Proof. This directly follows from the proof of Λ_{RO} in [5] as we mainly make use of π_{PDL} and π_{DLEQ}. For the sake of completeness, we give the proof as follows:

- **Correctness**: Assume the dealer D and parties $\{P_i\}_{i=1}^n$ follow the protocol. Given a setup from initial phase of the protocol for an input 1^λ we have $(y_{-\ell+1}, \ldots, y_n, (d, z(X)))$ from the sharing phase of the Λ_{RO}^{indv} where $y_j = h_0^{f_j}$ for $-\ell+1 \leq j \leq 0$ and $y_i = PK_i^{f_i}$ for $1 \leq i \leq n$, d is an output from the random oracle $\mathcal{H}$ for the input $(y_{-\ell+1}, \ldots, y_n, c_{-\ell+1}, \ldots, c_n)$ and $z(X) = r(x) + df(X)$ is a $t+\ell-1$-degree polynomial such that $f(X)$ is the secret polynomial, $r(X)$ is the polynomial (also secret to D) chosen uniformly at random, $c_j = h_0^{r(j)}$ for $-\ell+1 \leq j \leq 0$ and $c_i = PK_i^{r(i)}$ for $1 \leq i \leq n$.
Now with $(y_{-\ell+1} \ldots, y_n, (d, z(X)))$, a verifier checks that the following part of the verification phase evaluates to d when additionally given with the commitment key of a target entity for the secrets and n public keys for the shares:

$$\begin{aligned}&\mathcal{H}(y_{-\ell+1} \ldots, y_n, \frac{{h_0}^{z(-\ell+1)}}{y_{-\ell+1}^d}, \ldots, \frac{h_0^{z(0)}}{y_0^d}, \frac{PK_1^{z(1)}}{y_1^d}, \ldots, \frac{PK_n^{z(n)}}{y_n^d})\\ =&\mathcal{H}(y_{-\ell+1}, \ldots, y_n, h_0^{r(-\ell+1)}, \ldots, h_0^{r(0)}, PK_1^{r(1)}, \ldots, PK_n^{r(n)}).\end{aligned}$$

Moreover, the reconstruction phase always yields same secrets $\{g^{f_j}\}_{j=-\ell+1}^0$ in both the approaches. Explicitly, it is clear in the optimistic phase that yields $\{g^{f_j}\}_{j=-\ell+1}^0$ after one checks $\{y_j = h_0^{f_j}\}$ for $-\ell+1 \leq j \leq n$, when given $(g, h_0, \{y_j, f_j\}_{j=-\ell+1}^0)$. The Pessimistic case also yields $\{g^{f_j}\}_{j=-\ell+1}^0$ which inherently is the reconstruction step from the PVSS Π_S [1]. In essence, we just proved that if the dealer and parties follow the protocol, then verification step returns true and the Reconstruction phase returns the actual secrets.

- **Verifiability**: We need to show that if verify algorithm returns true then for $1 \leq i \leq n$, y_i is a valid encryption of i'th share of the secrets $\{g^{f_j}\}_{j=-\ell+1}^0$ which is intended to party P_i for $1 \leq i \leq n$ and for $-\ell+1 \leq j \leq 0$, y_j is the valid commitment of the secret g^{f_j} and *reconstruct* algorithm returns $\{g^{f_j}\}_{j=-\ell+1}^0$ irrespective of the approach with high probability. The trick to do that is to leverage the special soundness property of underlying proof system.
Assuming verification phase outputs true, consider two acceptable transcripts $(c_{-\ell+1}, \ldots, c_n, d, z(X)), (c_{-\ell+1}, \ldots, c_n, d', z'(X))$ of the interactive version of the NIZK argument for PDL problem output by the dealer during the sharing phase as follows.

$$\begin{aligned}h_0^{z(j)} = c_j y_j^d, h_0^{z'(j)} = c_j y_j^{d'}, \text{ for } -\ell+1 \leq j \leq 0;\\ PK_i^{z(i)} = c_i y_i^d, PK_i^{z'(i)} = c_i y_i^{d'}, \text{ for } 1 \leq i \leq n.\end{aligned}$$

this implies,

$$\begin{aligned}h_0^{z(j)-z'(j)} = y_j^{d-d'} \iff h_0^{\frac{z(j)-z'(j)}{d-d'}} = y_j, \text{ for } -\ell+1 \leq j \leq 0;\\ PK_i^{z(i)-z'(i)} = y_i^{d-d'} \iff PK_i^{\frac{z(i)-z'(i)}{d-d'}} = y_i, \text{ for } 1 \leq i \leq n.\end{aligned}$$

Now if all $n + \ell$ checks pass then it means that the given commitments of secrets and encryptions of their shares are valid with high probability. Moreover, any set $\mathcal{Q}$ of $t + \ell$ honest parties can decrypt the encryptions of their shares to reconstruct secrets via pessimistic approach(because $n \geq 2t + \ell$), then the following is obtained,

$$g^{\frac{z(i)-z'(i)}{d-d'}} = F_i \text{ for } i \in \mathcal{Q}, |\mathcal{Q}| = t + \ell.$$

After the DLEQ proofs π_{DLEQ} are checked, as with high probability $z(X), z'(X)$ are exactly $t+\ell-1$ degree polynomial, an extractor having access to the two acceptable transcripts can construct the secrets from $\{f_i = \frac{z(i)-z'(i)}{d-d'}\}_{i\in\mathcal{Q}}$, which are *exponents* of ℓ evaluations of the $t + \ell - 1$-degree polynomial $f(X) = \frac{z(X)-z'(X)}{d-d'}$. This implies that pessimistic approach yields the secrets $\{g^{f_j}\}_{j=-\ell+1}^{0}$ with high probability.
Similarly, the optimistic approach also yields the actual secrets $\{g^{f_j}\}_{j=-\ell+1}^{0}$ with high probability as a consequence of special soundness of the NIZK argument for PDL problem deployed in our 3PVSS.

- **IND1-Secrecy**: We will prove that our scheme will not leak any information about the secrets to any static computationally bounded adversary $\mathcal{A}$ who corrupts at most t parties and has access to their secret keys, all public information except the secret key of target entity, i.e., SK_0, assuming the DDH assumption is true.
In a nutshell, let $s_0 = (g^{f_{-\ell+1}}, \ldots, g^{f_0})$ and $s_1 = (g^{f'_{-\ell+1}}, \ldots, g^{f'_0})$ be the tuples of secrets in $\mathbb{Z}_q$ chosen at random by the IND1-Secrecy challenger. The challenger runs the setup phase of our 3PVSS scheme Λ_{RO}^{indv} to obtain the h_0 for committing to the secrets and the public keys $\{PK_i\}_{i=1}^{n}$ to encrypt the shares meant for shareholders P_i for $1 \leq i \leq n$. The challenger runs $share(n, t + \ell - 1, s_0, \{PK_i\}_{i=1}^{n}, h_0)$ and obtains $(y_{-\ell+1}, \ldots, y_n, \pi_{3PVSS}^{share})$ as the committed secrets and encrypted shares, and obtains the proof of correctness of the sharing phase. The challenger chooses $b \in \{0, 1\}$ uniformly at random and sends $s_b, (y_{-\ell+1}, \ldots, y_n, \pi_{3PVSS}^{share})$ along with the public information generated in the sharing phase to $\mathcal{A}$. *Without loss of generality, let $f_i = f'_i$ for $-\ell + 1 \leq i \leq -1$, i.e., $\mathcal{A}$ is trying to distinguish between two tuples of secrets where they may differ only in last position.*
By contradiction, assume $\mathcal{A}$ can break the IND1-Secrecy property of our scheme. Then we will show that there exists an adversary $\mathcal{B}$ who can break the DDH assumption using $\mathcal{A}$ as a subroutine. Also, without loss of generality assume $\mathcal{A}$ corrupts first t shareholders, i.e., corrupts $P_1, \ldots, P_t$ so it knows their secret keys.
Let $\mathcal{B}$ be an adversary who is given a DDH instance $(h, h^\alpha, h^\beta, h^\gamma)$, where $\alpha, \beta, \gamma \in \mathbb{Z}_q$ with α and β being non-zero. Now $\mathcal{B}$ using $\mathcal{A}$ simulates the IND1 game $\mathcal{A}$ as follows:
  - $\mathcal{B}$ sets $g = h^\alpha$.
  - For $t+1 \leq i \leq n$, $\mathcal{B}$ selects $u_i \in \mathbb{Z}_q$ (implicitly defines $s_i = \frac{u_i}{\alpha}$) uniformly at random and sends $PK_i = h^{u_i}$ to $\mathcal{A}$.

- $\mathcal{B}$ selects $u_0 \in \mathbb{Z}_q$ uniformly at random and sets $h_0 = h^{u_0}$ as the commitment key for the secrets.
- For $1 \leq i \leq t$, $\mathcal{A}$ selects $SK_i \in \mathbb{Z}_q$ uniformly at random and sends $PK_i = g^{SK_i}$ to $\mathcal{B}$ (challenger in the perspective of $\mathcal{A}$).
- For $1 \leq i \leq t$ and $-\ell + 1 \leq j \leq -1$, $\mathcal{B}$ selects $f_i, f_j \in \mathbb{Z}_q$ uniformly at random, sets $v_i = h^{f_i}$,$v_j = h^{f_j}$ and $y_i = PK_i^{f_i}$, $y_j = h_j^{f_j}$. Finally, for $i = 0$, $\mathcal{B}$ sets $v_0 = h^{\beta}$ and $y_0 = v_0^{u_0}$, i.e., internally she assumes $f_0 = \beta$ but note that she does not know f_0 explicitly.
- For $t + 1 \leq i \leq n$, $\mathcal{B}$ generates $v_i = h^{p(i)}$ via lagrange interpolation in the exponent, where $p \in \mathbb{Z}_q[X]_{t+\ell-1}$ which is determined by the $t + \ell$ evaluations, namely, $p(0) = f_0 = \beta$, $p(i) = f_i$ for $-\ell + 1 \leq i \leq -1$ and $1 \leq i \leq t$. After obtaining v_i, $\mathcal{B}$ sets $y_i = v_i^{u_i}$ for $t + 1 \leq i \leq n$.
- Now that $\mathcal{B}$ has $h_0, PK_1, \ldots, PK_n, y_{-\ell+1}, \ldots, y_n$, as a simulator she samples a random $t + \ell - 1$ degree polynomial $z'(X)$ and sets $c'_j = \frac{h_0^{z'(j)}}{y_j^d}$ for $-\ell + 1 \leq j \leq 0$ and $c'_i = \frac{PK_i^{z'(i)}}{y_i^d}$ for $1 \leq i \leq n$, when the challenge value d in the interactive version of the proof scheme is given.
- $\mathcal{B}$ sets h^{γ} as the last element in s_b.
- Finally, $\mathcal{B}$ sends $\{y_{-\ell+1}, \ldots, y_n, c'_{-\ell+1}, \ldots, c'_n, d, z'(X)\}$ along with s_b and public information to $\mathcal{A}$, simulating the role of a IND1-Secrecy challenger.
- $\mathcal{A}$ outputs the bit b' as its guess for b. If $b' = 0$ then $\mathcal{B}$ guesses $\gamma = \alpha.\beta$, otherwise γ is guessed to be a random element in $\mathbb{Z}_q$.

□

We discuss the performance of the new 3PVSS scheme Λ_{RO}^{indv} later in this section. It is worth noting that Λ_{RO}^{indv} can be viewed as a generalization of the PPVSS scheme Λ_{RO} proposed in [5]. In particular, their construction corresponds to the special case of Λ_{RO}^{indv} when $\ell = 1$. Because of this relationship, we believe that all applications discussed in [5] can directly benefit from our proposed protocol.

3.3 A Packed Pre-Constructed PVSS with Single Commitment

In this section, we present our second practical 3PVSS construction Λ_{RO}^{single} in which the dealer commits to all shared secrets using a *single* commitment. To this end, we first introduce a new NIZK argument of knowledge for a specific language that will be useful for our 3PVSS construction. We then use a variant of this argument as a subroutine to build the protocol.

As in the first 3PVSS scheme Λ_{RO}^{indv}, the dealer shares a vector of secrets $\vec{s}$ of length ℓ using packed Shamir secret sharing and encrypts the resulting shares under the parties' public keys. However, instead of publishing ℓ individual commitments, the dealer computes a single commitment that binds all ℓ secrets simultaneously. The dealer then generates and broadcasts a NIZK proof π_{3PVSS} demonstrating the well-formedness of both the encrypted shares and the committed secrets. This design significantly reduces the communication overhead

of the protocol. In particular, since only a single commitment is broadcast, the communication complexity decreases from $O(n+\ell)$ in Λ_{RO}^{indv} to $O(n)$. As a result, this construction can outperform the previous 3PVSS scheme in scenarios where communication efficiency is critical. Nevertheless, the scheme with individual commitments retains certain advantages in certain applications, as we illustrate in the following section.

A NIZK Argument of Knowledge for Λ_{RO}^{single}. Consider the set $\{g_j\}_{j=-\ell+1}^{0}$ containing ℓ distinct generators of the prime order q cyclic group $\mathbb{G}$ different from g with g also being a generator of $\mathbb{G}$, where $\ell < \varphi(q)$ and $\varphi(q)$ is the total number of distinct generators of the cyclic group $\mathbb{G}$. Consider the following relation for some polynomial $f \in \mathbb{Z}_q[X]_{t+\ell-1}$ with $2t + \ell \leq n$:

$$R_{mPDL} = \{(g_{-\ell+1}, \ldots, g_0, g, x_{-\ell+1}, \ldots, x_n, F_0, \ldots, F_n), f(X) : F_0 = \prod_{j=-\ell+1}^{0} g_j^{f(x_j)}, F_i = g^{f(x_i)}, 1 \leq i \leq n\}, \quad (2)$$

where all x_i's are distinct in $\mathbb{Z}_q$. The R_{mPDL} is based on *modified*-PDL (inspired from the PDL in [4]) which is defined in the following.

Definition 3 (modified Polynomial Discrete Logarithm Problem). *Let $\{g_j : g_j \neq g\}_{j=-\ell+1}^{0}$ be a set of random distinct generators for the prime order q cyclic group $\mathbb{G}$ generated by g. Given $F_0, \ldots, F_n$ and distinct elements $x_{-\ell+1}, \ldots, x_n$ in $\mathbb{Z}_q$, find a polynomial $f \in \mathbb{Z}_q[X]$ with degree at most $t + \ell - 1$, where $F_0 = \prod_{j=-\ell+1}^{0} g_j^{f(x_j)}$ and $F_i = g^{f(x_i)}$ for $1 \leq i \leq n$ with $2t + \ell \leq n$.*

In other words, an algorithm $\mathcal{A}$ is said to have an advantage ϵ in solving modified-PDL if

$$Pr[\mathcal{A}(x_{-\ell+1}, \ldots, x_n, g, g_{-\ell+1}, \ldots, g_0, \prod_{j=-\ell+1}^{0} g_j^{f(x_j)}, g^{f(x_1)}, \ldots, g^{f(x_n)})] \geq \epsilon,$$

where $f \in \mathbb{Z}_q[X]$ is at most a $t + \ell - 1$ degree polynomial with $2t + \ell \leq n$ and the probability is over distinct generators $g, g_{-\ell+1}, \ldots, g_0$ of $\mathbb{G}$ chosen at random and distinct $x_{-\ell+1}, \ldots, x_n$ elements in $\mathbb{Z}_q$.

On Hardness of modified-PDL Problem. It is intuitive to observe that, similarly to the original PDL problem [4], the *modified*-PDL problem reduces to the Discrete Logarithm (DL) problem. For completeness, we include a proof in Appendix A. Now consider the language of R_{mPDL} as follows:

$$L_{g_{-\ell+1},\ldots,g_0}^{mPDL} = \{(g, x_{-\ell+1}, \ldots, x_n, F_0, \ldots, F_n) : \exists f \in \mathbb{Z}_q[X]_{t+\ell-1}, [(g_{-\ell+1}, \ldots, g_0, g, x_{-\ell+1}, \ldots, x_n, F_0, \ldots, F_n), f(X)] \in R_{mPDL}\}.$$

We now present an efficient NIZK Argument of Knowledge (AoK) π_{mPDL}^{AoK} for the aforementioned relation R_{mPDL} in the Figure 5.

Non-Interactive Zero-Knowledge Argument π^{AoK}_{mPDL}

Let $(g, x_{-\ell+1}, \ldots, x_n, F_0, \ldots, F_n) \in L^{mPDL}_{g_{-\ell+1},\ldots,g_0}$ with its corresponding witness being $f \in \mathbb{Z}_q[X]_{\leq t+\ell-1}$ where $L^{mPDL}_{g_{-\ell+1},\ldots,g_0}$ is the language defined by the relation R_{mPDL} and $\mathcal{H}$ be a random oracle (RO).

Prover: Given the witness $f(X)$, samples $r(X) \in_R \mathbb{Z}_q[X]_{t+\ell-1}$ uniformly at random and sets $\Gamma_0 = \prod_{j=-\ell+1}^{0} g_j^{r(x_j)}$ and $\Gamma_i = g^{r(x_i)}$ for $1 \leq i \leq n$. Then, sets $d \leftarrow \mathcal{H}(F_0, \ldots, F_n, \Gamma_0, \ldots, \Gamma_n)$, where $\mathcal{H}$ is the random oracle. Finally, sets $z(X) \equiv r(X) + df(X) \pmod q$ and returns $\pi := (d, z(X))$.

Verifier: Given the public statement and the proof π, first checks if $z(X)$ is a (at most) $t + \ell - 1$ degree polynomial in $\mathbb{Z}_q[X]$. If so, it checks if

$$d \leftarrow \mathcal{H}(F_0, \ldots, F_n, \frac{\prod_{j=-\ell+1}^{0} g_j^{z(x_j)}}{F_0^d}, \frac{g^{z(x_1)}}{F_1^d}, \ldots, \frac{g^{z(x_n)}}{F_n^d}).$$

If first two steps are correct then they output *true*, otherwise *false*.

Fig. 5: An Efficient NIZK AoK for the *modified*-PDL

Theorem 2 (A NIZK Argurement of Knowledge for R_{mPDL}). *Consider $(g, x_{-\ell+1}, \ldots, x_n, F_0, \ldots, F_n) \in L^{mPDL}_{g_{-\ell+1},\ldots,g_0}$, where $L^{mPDL}_{g_{-\ell+1},\ldots,g_0}$ is the language defined by the relation R_{mPDL}, with its corresponding witness being $f \in \mathbb{Z}_q[X]_{\leq t+\ell-1}$. Assuming modified-PDL is computationally hard, for $2t + \ell \leq n$, the protocol π^{AoK}_{mPDL} is a NIZK AoK for R_{mPDL} in the RO model.*

Proof. The corresponding proof is similar to the proof of theorem given in [1]. Formally, we will prove the security of the interactive setting (i.e., without *RO* being used) and then using Fiat-Shamir transform it can be extended for the non-interactive setting in the *RO* model.

- ***Correctness***: If both prover and verifier are honest, then for $1 \leq i \leq n$ we have

$$\begin{aligned} g^{z(x_i)} &= g^{r(x_i)+df(x_i)} \\ &= g^{r(x_i)}(g^{f(x_i)})^d \\ &= \Gamma_i F_i^d, \end{aligned} \tag{3}$$

and

$$\begin{aligned} \prod_{j=-\ell+1}^{0} g_j^{z(x_j)} &= \prod_{j=-\ell+1}^{0} g_j^{r(x_j)+df(x_j)} \\ &= (\prod_{j=-\ell+1}^{0} g_j^{r(x_j)})(\prod_{j=-\ell+1}^{0} g_j^{f(x_j)})^d \\ &= \Gamma_0 F_0^d. \end{aligned} \tag{4}$$

The aforementioned equations imply that verification returns *true* and honest verifier accepts the honest prover.

- ***Special Soundness***: Let $(\Gamma_0, \ldots, \Gamma_n, d, z(X)), (\Gamma_0, \ldots, \Gamma_n, d', z'(X))$ be two acceptable transcripts where response polynomials will differ as a consequence of different challenge values. For $1 \leq i \leq n$ we have the following from equation 3:

$$g^{z(x_i)} = \Gamma_i F_i^d, g^{z'(x_i)} = \Gamma_i F_i^{d'};$$

which implies

$$g^{z(x_i)-z'(x_i)} = F_i^{d-d'} \iff g^{\frac{z(x_i)-z'(x_i)}{d-d'}} = F_i, \tag{5}$$

[*if and only if because* $d-d'$ *is always invertible in modulo prime* q *whenever* $d \neq d' \pmod q$]. Similarly, we have the following from equation 4:

$$\prod_{j=-\ell+1}^{0} g_j^{z(x_j)} = \Gamma_0 F_0^d, \prod_{j=-\ell+1}^{0} g_j^{z'(x_j)} = \Gamma_0 F_0^{d'};$$

implying

$$\prod_{j=-\ell+1}^{0} g_j^{z(x_j)-z'(x_j)} = F_0^{d-d'} \iff \prod_{j=-\ell+1}^{0} g_j^{\frac{z(x_j)-z'(x_j)}{d-d'}} = F_0. \tag{6}$$

As we have $n \geq 2t+\ell$, information from equation 5 is enough for an extractor to extract a unique witness polynomial f in the *exponent* for the given statement in $R_{mod-PDL}$ as it implies that $f_i = \frac{z(x_i)-z'(x_i)}{d-d'}$ for $1 \leq i \leq n$. More explicitly, as $z(X)$ is a $t+\ell-1$ degree polynomial with high probability in $\mathbb{Z}_q[X]$, an extractor $\mathcal{E}$ can construct the unique $t+\ell-1$-degree polynomial $f \in \mathbb{Z}_q[X]$ (in the *exponent*), being the desired witness (resp. solution) for a given statement in $R_{mod-PDL}$ relation (resp. *modified*-PDL problem), from any $t+\ell$ evaluation points in $\{f_i\}_{i=1}^n$ whenever $n \geq 2t+\ell$.

- ***Honest Verifier Zero Knowledge (HVZK)***: Given the statement $(g, x_{-\ell+1}, \ldots, x_n, F_0, \ldots, F_n) \in L^{mod-PDL}_{g_{-\ell+1},\ldots,g_0}$ and a challenge value d, a simulator $\mathcal{S}$ can choose a polynomial $z' \in \mathbb{Z}_q[X]_{t+\ell-1}$ uniformly at random and sets $\Gamma'_0 = \frac{\prod_{j=-\ell+1}^{0} g_j^{z'(x_j)}}{F_0^d}$ and $\Gamma'_i = \frac{g_i^{z'(x_i)}}{F_i^d}$ for $1 \leq i \leq n$. Now, $\mathcal{S}$ returns $(\Gamma'_0, \ldots, \Gamma'_n, z'(X))$ as the simulated proof. As $z(X)$ is a random degree $t+\ell-1$-polynomial in $\mathbb{Z}_q[X]$ and $z'(X)$ is chosen uniformly at random, the simulated proof of $\mathcal{S}$ is indistinguishable from the real one.

As the interactive scheme is public coin, satisfies *completeness*, (computational) *Special Soundness* and (computational) *HVZK*, then in the random oracle (*RO*) model, using Fiat-Shamir transform [16], it can be turned into a NIZK Argument of Knowledge for $R_{mod-PDL}$ (defined in equation 2). □

Packed Pre-Constructed PVSS Scheme Λ_{RO}^{single}

Initial: All parties $\{P_i\}_{i=1}^n$ and dealer D agree on the prime field $\mathbb{Z}_q$, a group $(\mathbb{G}, \times)$ of order q with a generator g, random oracle $\mathcal{H}$. Also, each party P_i registers their public key PK_i in the public ledger and all agree on a set of commitment keys, $\{h_j\}_{j=-\ell+1}^0$.

Share: Given $(n, t, \vec{s}, \{PK_i\}_{i=1}^n, \{h_j\}_{j=-\ell+1}^0)$, D returns $(y_0, \{y_i\}_{i=1}^n, \pi_{3PVSS})$ as follows:

- Dealer D samples a $t+\ell-1$-degree polynomial $f \in \mathbb{Z}_q[X]$ uniformly at random and sets $\{g^{f_j} : f_j = f(j)\}_{j=-(\ell-1)}^0$ as the set of all secrets, where $\vec{s} := (f_{-(\ell-1)}, \ldots, f_{-1}, f_0)$.
- For each $1 \le i \le n$, D uses PK_i and $f_i = f(i)$ and computes encrypted values $y_i = PK_i^{f_i}$. D also computes the commitments to secrets $\{g^{f_j}\}_{j=-(\ell-1)}^0$ as $y_0 = \prod_{j=-\ell+1}^0 h_j^{f_j}$.
- D uses the modified-PDL proof scheme from Fig. 5 and generate π_{3PVSS} as follows:
  - Samples a $t+\ell-1$-degree polynomial $r \in \mathbb{Z}_q[X]$ uniformly at random and computes $c_0 = \prod_{j=-\ell+1}^0 h_j^{r(j)}$ and $c_i = PK_i^{r(i)}$ for $1 \le i \le n$.
  - Using $\mathcal{H}$, $d = \mathcal{H}(y_0, \ldots, y_n, c_0, \ldots, c_n)$ is computed.
  - Sets $z(X) = r(X) + df(X)$, hence $\pi_{3PVSS} = (d, z(X))$ is obtained.
- D broadcasts commitment y_0, encrypted shares $\{y_i\}_{i=0}^n$ and the proof $\pi_{3PVSS} = (d, z(X))$.

Verify: Given $(n, t, \{h_j\}_{j=-\ell+1}^0, y_0, \{PK_i, y_i\}_{i=1}^n, \pi_{3PVSS})$, the verifier acts as follows:

- Checks if $z(X)$ is a (at most) $t+\ell-1$-degree polynomial or not.
- Checks if $d = \mathcal{H}(y_0, y_1, \ldots, y_n, \frac{\prod_{j=-\ell+1}^0 h_j^{z(j)}}{y_0^d}, \frac{PK_1^{z(1)}}{y_1^d}, \ldots, \frac{PK_n^{z(n)}}{y_n^d})$.
- Outputs *true* if both of the above checks are satisfied, otherwise *false*.

Reconstruct: Reconstruction of the secrets $\{g^{f_j}\}_{j=-(\ell-1)}^0$ can be done by two approaches:

- **Optimistic:** First D publishes $\{f_j\}_{j=-(\ell-1)}^0$. Then given $g, \{h_j\}_{j=-\ell+1}^0, y_0$ parties check if $y_0 = \prod_{j=-\ell+1}^0 h_j^{f_j}$ and returns $\{g^{f_j}\}_{j=-(\ell-1)}^0$ if all checks pass, if not they return *false*.
- **Pessimistic:** Any qualified set $\mathcal{Q}$, where $|Q| \ge t+\ell$, of parties act as follows:
  - Each party $P_i \in \mathcal{Q}$ decrypts y_i using SK_i corresponding to PK_i and obtains g^{f_i} and then publishes it along with a DLEQ proof π_{DLEQ} to prove the correctness of decryption.
  - If $\mathcal{Q}$ consists at least $t+\ell$ honest parties, they can use Lagrange interpolation to compute the secrets $\{g^{f_j}\}_{j=-(\ell-1)}^0$ as follows:

$$g^{f_j} = \prod_{i \in \mathcal{Q}} (g^{f_i})^{\prod_{k \in \mathcal{Q}, k \ne i} \frac{-k-j}{i-j}} = g^{\sum_{i \in \mathcal{Q}} f_i \prod_{k \in \mathcal{Q}, k \ne i} \frac{-k-j}{i-j}}.$$

Fig. 6: Λ_{RO}^{single}: An efficient 3PVSS scheme with a single commitment

Construction of Λ_{RO}^{single} 3PVSS Scheme. Next, we present description of Λ_{RO}^{single} in Figure 4 which uses the NIZK AoK presented in Fig. 5 as a subroutine. We prove the security of new 3PVSS scheme in the following theorem.

Theorem 3. *Assuming mPDL problem and Decisional Diffie-Hellman (DDH) are computationally hard, Λ_{RO}^{single} is a secure 3PVSS scheme against any static computationally bounded adversary. corrupting up to t out of n parties, whenever $n \ge 2t+\ell$ with $\ell \ge 1$ being the total number of secrets.*

Proof. We prove the *Correctness, Verifiability* and *IND1-Secrecy* properties.

- **Correctness**: Assume the dealer D and parties $\{P_i\}_{i=1}^n$ follow the protocol. We show that verification returns *true* and reconstruction of secrets in both approaches will return the intended secrets. Given a setup from initial phase of the protocol for an input 1^λ we have $(y_0, y_1, \ldots, y_n, (d, z(X)))$ from the sharing phase of the Λ_{RO}^{single} where $y_0 = \prod_{j=-\ell+1}^0 h_0^{f_j}$, $y_i = PK_i^{f_i}$

for $1 \leq i \leq n$, d is an output from the random oracle $\mathcal{H}$ for the input $(y_0, \ldots, y_n, c_0, \ldots, c_n)$ and $z(X) = r(x) + df(X)$ is a $t+\ell-$degree polynomial such that $f(X)$ is a secret polynomial, $r(X)$ is the polynomial (also secret to D) chosen uniformly at random, $c_0 = \prod_{j=-\ell+1}^{0} h_0^{r(j)}$ and $c_i = PK_i^{r(i)}$ for $1 \leq i \leq n$.
Now with $(y_0, y_1, \ldots, y_n, (d, z(X)))$, a verifier does the following computation for verification using ℓ commitment (/encryption) keys:

$$\begin{aligned}
&\mathcal{H}(y_0, y_1, \ldots, y_n, \frac{\prod_{j=-\ell+1}^{0} h_j^{z(j)}}{y_0^d}, \frac{PK_1^{z(1)}}{y_1^d}, \ldots, \frac{PK_n^{z(n)}}{y_n^d}) \\
=&\mathcal{H}(y_0, y_1, \ldots, y_n, \prod_{j=-\ell+1}^{0} \frac{h_j^{r(j)+df(j)}}{h_j^{df(j)}}, \frac{PK_1^{r(1)+df(1)}}{PK_1^{df(1)}}, \ldots, \frac{PK_n^{r(n)+df(n)}}{h_n^{df(n)}}) \\
=&\mathcal{H}(y_0, y_1, \ldots, y_n, \prod_{j=-\ell+1}^{0} h_j^{r(j)}, PK_1^{r(1)}, \ldots, PK_n^{r(n)}).
\end{aligned}$$

We show that the reconstruction phase always yields $\{g^{f_j}\}_{j=-\ell+1}^{0}$ in both the approaches. Explicitly, it is clear in the optimistic phase that yields $\{g^{f_j}\}_{j=-\ell+1}^{0}$ after one checks $y_0 = \prod_{j=-\ell+1}^{0} h_0^{f_j}$ when given $(g, h_0, y_0, \{f_j\}_{j=-\ell+1}^{0})$. The Pessimistic case also yields $\{g^{f_j}\}_{j=-\ell+1}^{0}$ which inherently is the reconstruction step from the PVSS Π_S [1]. In essence, we just proved that if the dealer and parties follow the protocol, then verification step returns true and the Reconstruction phase returns the actual secrets.

- **Verifiability**: We will show that if verification algorithm returns true then with high probability y_i is a valid encryption of i'th share of the secrets $\{g^{f_j}\}_{j=-\ell+1}^{0}$ which is intended to party P_i for $1 \leq i \leq n$, y_0 is the commitment of ℓ secrets and reconstruct algorithm returns $\{g^{f_j}\}_{j=-\ell+1}^{0}$ irrespective of the two approaches.
Assuming the verification algorithm returns *true*, consider two acceptable transcripts $(c_0, \ldots, c_n, (d, z(X)))$ and $(c_0, \ldots, c_n, (d', z'(X)))$ of the interactive version of π_{share} with $d \neq d'$. Then we have the following from the verification step of π_{share}:

$$\prod_{j=-\ell+1}^{0} h_j^{z(j)} = c_0 y_0^d, \prod_{j=-\ell+1}^{0} h_j^{z(j)} = c_0 y_0^d,$$

which implies

$$\prod_{j=-\ell+1}^{0} h_j^{z(j)-z'(j)} = y_0^{d-d'}, \prod_{j=-\ell+1}^{0} h_j^{\frac{z(j)-z'(j)}{d-d'}} = y_0.$$

Also for $1 \leq i \leq n$ we have the following:

$$PK_i^{z(i)} = c_i y_i^d, PK_i^{z'(i)} = c_i y_i^{d'},$$

as a consequence we have,

$$PK_i^{z(i)-z'(i)} = y_i^{d-d'} \iff PK_i^{\frac{z(i)-z'(i)}{d-d'}} = y_i.$$

Now if all $n+1$ checks pass then it means that the given commitment of secrets and encryptions of their shares are valid with high probability. Moreover, any set $\mathcal{Q}$ of $t+\ell$ honest parties can decrypt the encryptions of their shares to reconstruct secrets via pessimistic approach(because $n \geq 2t+\ell$), then the following is obtained,

$$g^{\frac{z(i)-z'(i)}{d-d'}} = F_i \text{ for } i \in \mathcal{Q}, |\mathcal{Q}| = t+\ell.$$

After the DLEQ proofs π_{DLEQ} are checked, as with high probability $z(X), z'(X)$ are exactly $t+\ell-1$ degree polynomial, an extractor having access to the two acceptable transcripts can construct the secrets from $\{f_i = \frac{z(i)-z'(i)}{d-d'}\}_{i\in\mathcal{Q}}$, which are *exponents* of ℓ evaluations of the $t+\ell-1$-degree polynomial $f(X) = \frac{z(X)-z'(X)}{d-d'}$. This implies that pessimistic approach yields the secrets $\{g^{f_j}\}_{j=-\ell+1}^{0}$ with high probability.
Similarly, the optimistic approach also yields the actual secrets $\{g^{f_j}\}_{j=-\ell+1}^{0}$ with high probability as a consequence of special soundness of the NIZK argument for PDL problem deployed in our 3PVSS.

- **IND1-Secrecy**: We will prove that our scheme will not leak any information about the secrets to any static computationally bounded adversary $\mathcal{A}$ who corrupts at most t out of n parties, assuming the DDH assumption is true.
In a nutshell, let $s_0 = (g^{f_{-\ell+1}}, \ldots, g^{f_0})$ and $s_1 = (g^{f'_{-\ell+1}}, \ldots, g^{f'_0})$ be the tuples of secrets in $\mathbb{Z}_q$ chosen at random by the IND1-Secrecy challenger. The challenger runs the setup phase of our 3PVSS scheme Λ_{RO}^{single} to obtain the commitment keys $\{h_j\}_{j=-\ell+1}^{0}$ for committing to the secrets and the public keys $\{PK_i\}_{i=1}^{n}$ to encrypt the shares meant for shareholders P_i for $1 \leq i \leq n$. The challenger runs $share(n, t+\ell-1, s_0, \{h_j\}_{j=-\ell+1}^{0}, \{PK_i\}_{i=1}^{n})$ and obtains $(y_0, \ldots, y_n, \pi_{3PVSS}^{share})$ as the committed secrets and their encrypted shares, along with the proof of correctness of the sharing phase. The challenger chooses $b \in \{0,1\}$ uniformly at random and sends $s_b, (y_0, \ldots, y_n, \pi_{3PVSS}^{share})$ along with the public information generated in the sharing phase to $\mathcal{A}$. *Without loss of generality, let $f_i = f'_i$ for $-\ell+1 \leq i \leq -1$, i.e., $\mathcal{A}$ is trying to distinguish between two tuples of secrets where they may differ only in last position.*
By contradiction, assume $\mathcal{A}$ can break the IND1-Secrecy property of our scheme. Then we will show that there exists an adversary $\mathcal{B}$ who can break the DDH assumption using $\mathcal{A}$ as a subroutine. Also, without loss of generality assume $\mathcal{A}$ corrupts first t shareholders, i.e., corrupts $P_1, \ldots, P_t$ so it knows their secret keys.
Let $\mathcal{B}$ be an adversary who is given a DDH instance $(h, h^\alpha, h^\beta, h^\gamma)$, where $\alpha, \beta, \gamma \in \mathbb{Z}_q$ with α and β being non-zero. Now $\mathcal{B}$ using $\mathcal{A}$ simulates the IND1 game $\mathcal{A}$ as follows:

- $\mathcal{B}$ sets $g = h^{\alpha}$.
- For $t+1 \leq i \leq n$ and $-\ell+1 \leq j \leq 0$, $\mathcal{B}$ selects $u_i, u_j \in \mathbb{Z}_q$ (implicitly defines $s_i = \frac{u_i}{\alpha}$) uniformly at random and sends $PK_i = h^{u_i}$, $h_j = h^{u_j}$ to $\mathcal{A}$.
- For $1 \leq i \leq t$, $\mathcal{A}$ selects $SK_i \in \mathbb{Z}_q$ uniformly at random and sends $PK_i = g^{SK_i}$ to $\mathcal{B}$ (challenger in the perspective of $\mathcal{A}$).
- For $1 \leq i \leq t$, $\mathcal{B}$ selects $f_i \in \mathbb{Z}_q$ uniformly at random, sets $v_i = h^{f_i}$ and $y_i = PK_i^{f_i}$.
- For $-\ell+1 \leq j \leq -1$, $\mathcal{B}$ selects $f_j \in \mathbb{Z}_q$ uniformly at random, sets $v_j = h^{f_j}$. Also, $\mathcal{B}$ sets $v_0 = h^{\beta}$. Now that she has $\{v_j, u_j\}_{j=-\ell+1}^{0}$, she sets $y_0 = \prod_{j=-\ell+1}^{0} v_j^{u_j}$. Observe that $\mathcal{B}$ assumes $f_0 = \beta$ but does not know it explicitly.
- For $t+1 \leq i \leq n$, $\mathcal{B}$ generates $v_i = h^{p(i)}$ via lagrange interpolation in the exponent, where $p \in \mathbb{Z}_q[X]_{t+\ell-1}$ which is determined by the $t+\ell$ evaluations, namely, $p(0) = f_0 = \beta$, $p(i) = f_i$ for $-\ell+1 \leq i \leq -1$ and $1 \leq i \leq t$. After obtaining v_i, $\mathcal{B}$ sets $y_i = v_i^{u_i}$ for $t+1 \leq i \leq n$.
- Now that $\mathcal{B}$ has $\{h_{-\ell+1}, \ldots, h_0, PK_1, \ldots, PK_n, y_0, \ldots, y_n\}$, as a simulator she samples a random $t+\ell-1$ degree polynomial $z'(X)$ and sets $c'_i = \frac{PK_i^{z'(i)}}{y_i^d}$ for $1 \leq i \leq n$ and $c'_0 = \frac{\prod_{j=-\ell+1}^{0} h_j^{z'(j)}}{y_0^d}$ when the challenge value d in the interactive version of the proof scheme is given.
- $\mathcal{B}$ sets h^{γ} as the last element in s_b.
- Finally, $\mathcal{B}$ sends $\{y_0, \ldots, y_n, c'_0, \ldots, c'_n, d, z'(X)\}$ along with s_b and public information to $\mathcal{A}$, simulating the role of a IND1-Secrecy challenger.
- $\mathcal{A}$ outputs the bit b' as its guess for b. If $b' = 0$ then $\mathcal{B}$ guesses $\gamma = \alpha.\beta$, otherwise γ is guessed to be a random element in $\mathbb{Z}_q$. □

3.4 Efficiency of New 3PVSS Schemes

Heuristic cost comparisons of both 3PVSS schemes, Λ_{RO}^{indv} and Λ_{RO}^{single}, is given in Appendix B.1. *Note that, the candidate 3PVSS schemes may not be comparable in terms of applications, we only made heuristic comparisons for completion.*

4 Multi-Candidate E-Voting Protocol Based on Λ_{RO}^{indv}

In 1999, Schoenmakers [23] proposed a universally verifiable binary e-voting protocol based on (P)PVSS. In his construction, voters use a PPVSS scheme to submit encrypted votes together with proofs of validity, while a set of tallying authorities jointly reconstructs the final outcome. In this protocol, the verification phase requires $O(mn^2)$ exponentiations, where n and m denote the number of talliers and voters, respectively. Recently, Baghery et al. [5] revisited Schoenmakers' protocol and improved its efficiency by introducing a more efficient PPVSS construction. They also observed that this application inherently requires a PPVSS scheme. Their approach reduces the verification cost to

$O(mn)$ exponentiations, significantly improving scalability for elections with a large number of talliers.

However, both the original protocol [23] and its improved version [5] are designed for binary elections, where voters must choose between two candidates. Schoenmakers outlined a generic approach for extending such protocols to the ℓ-candidate setting by running the binary protocol ℓ times. Unfortunately, this approach increases both computation and communication costs by a factor of ℓ. In particular, for the current state-of-the-art protocol [5], the voters' computation and communication costs become $O(n\ell)$ exponentiations and $O(n\ell)$ group elements, respectively, while the ballot verification cost grows to $O(mn\ell)$ exponentiations. In large-scale elections with many talliers and candidates, these costs can become impractical. In this section, we show how our proposed 3PVSS protocol Λ_{RO}^{indv} can be used to efficiently extend the binary e-voting protocol to the ℓ-candidate setting. Compared to Schoenmakers' generic approach, our construction significantly reduces the computational and communication overhead. In particular, the voters' computation and communication costs decrease to $O(n+\ell)$ exponentiations and $O(n+\ell)$ group elements, respectively, while

An Efficient Universally Verifiable Multi-Candidate E-Voting Protocol

Initialization: The talliers $A_1, \ldots, A_n$ run the initialization phase of Λ_{RO}^{indv} and agree upon public keys $\{PK_i = G^{SK_i}\}_{i=1}^n$, a single commitment key h_0, and public indexes for ℓ candidates.

Ballot Casting: To vote $\bar{v} \in \{0,1\}^\ell$, a voter does the following:

- Samples $\bar{s} \in \mathbb{Z}_q^\ell$ uniformly at random.
- Using all the agreed upon public keys of talliers and the commitment key, they run Λ_{RO}^{indv} to secret share $\bar{s}$ and the associated (publicly verifiable) proof, π_{3PVSS}, amongst the talliers.
- For $j = -(\ell-1), \ldots, 0$: computes the value $U_j = G^{s_j + v_j}$ and using the protocol given in App. 2.3 constructs a well-formedness proof π_{U_j}, showing that indeed $v_j \in \{0,1\}$, without revealing any information on v_j. The proof π_{U_j} refers to the value of $y_j = h_0^{s_j}$ which is published as part of π_{3PVSS} in the 3PVSS Λ_{RO}^{indv} sharing phase, and show that

$$\log_G U_j = \log_{h_0} y_j \ \vee \ \log_G U_j = 1 + \log_{h_0} y_j$$

The ballots for voter V consists of the output of the 3PVSS sharing phase (which consists of encryption of shares and π_{3PVSS}, the value $\{U_j\}_{j=-(\ell-1)}^0$ (which can be seen as an encryption of the ℓ votes), and proofs $\{\pi_{U_j}\}_{j=-(\ell-1)}^0$ (which proves the well-formedness of $\{U_j\}_{j=-(\ell-1)}^0$).

Ballot Verification: Once voters submit their votes, the Bulletin board verifies the publicly verifiable proofs, π_{3PVSS} and $\{\pi_{U_j}\}_{j=-(\ell-1)}^0$, given by each voter.

Tallying: Let $V_1, \ldots, V_m$ be the honest voters. The talliers do the following:

- Each tallier A_i accumulates their shares obtained from the accepted voters to compute $\bar{Y_i}^* := \prod_{k=1}^m Y_{ik}$, which is the secret share of $\sum_{k=1}^m s_{ik}$ (that's to homomorphic property of the 3PVSS scheme).
- Then A_i, using $\bar{Y_i}^*$ applies the reconstruction protocol of Λ_{RO}^{indv} to obtain $\bar{S}^* := \bar{G}^{\sum_{k=1}^m \bar{s_j}}$, where $\bar{G} = (G, \ldots, G)$.
- Using $\{\bar{U}_k\}_{k=1}^m$ they compute $\bar{U}^* = \prod_{k=1}^m \bar{U}_k = \bar{G}^{\sum_{k=1}^m \bar{s_k} + \bar{v_k}}$. Then, using $\bar{S}^*$ and $\bar{U}^*$, they compute $\bar{V}^* = \frac{\bar{U}^*}{\bar{S}^*} = \bar{G}^{\sum_{k=1}^m \bar{v_k}}$.
- Given $\bar{V}^*$, they can compute the vector of $\bar{V} = \sum_{k=1}^m \bar{v_k}$. Let $\bar{V} := (V_{-(\ell-1)}, \ldots, V_0)$.
- Finally, the winner is determined by finding the index of $\max\{V_j\}$ for $j = -(\ell-1), \ldots, 0$.

Fig. 7: A multi-candidate e-voting protocol based on Λ_{RO}^{indv}, for $n \geq 2t + \ell$.

the ballot verification cost becomes $O(m(n+\ell))$ exponentiations. This efficiency improvement comes at the cost of reduced robustness. Specifically, if ℓ denotes the number of candidates and t out of n election authorities may be corrupted, then the condition $n \geq 2t + \ell$ must hold for the protocol to operate correctly. Our proposed ℓ-candidate e-voting protocol is described in Figure 7.

4.1 Security analysis

The new multi-candidate e-voting protocol is based on the 3PVSS Λ_{RO}^{indv} and a NIZK proof for binary vote, and thus its security directly follows from the security of Λ_{RO}^{indv} and $\{\pi_{U_j}\}_{j=-(\ell-1)}^{0}$. Moreover, double voting can easily checked in *Ballot Verification* phase as anonymity is not a requirement in our scheme. As a consequence of using 3PVSS, this e-voting protocol remains secure as long as the *secret information* about the commitment key, which is used to commit to the secrets, is unknown to the adversary.

4.2 Cost analysis

The main variable in the e-voting scheme is the number of voters m, hence, that the download size of the talliers is linear in the number of voters, m. In tallying phase, the main computation cost bore by the talliers comes from the reconstruction of sum of the secrets in the exponents, which is $5(t+\ell)\mathbb{E}_x + t\ell\mathbb{G}$, and multiplying their ciphers to obtain the shares of the sum of secrets, which additionally requires them to do $m\ell$ $\mathcal{M}_{\mathbb{G}}$. See Table 1 for the details.

Table 1: Communication cost of m voters and download size of n talliers, $\mathbb{G}$ = group of order q and $\mathbb{Z}_q$ = modular group of order q, where q is a large prime

Voter Communication	**Tallier download size**
$(n+5\ell)\mathbb{G} + (t+5\ell+1)\mathbb{Z}_q$	$[m(n+5\ell)+n+1]\mathbb{G}+m(t+5\ell+1)\mathbb{Z}_q$

Note that, the binary proof $\pi_{\bar{U}}$ is run (see Appendix 2.3 and Section 2.1 in [5] for more details) for each component of $\bar{U}$, where all components of $\bar{U}$ are independent of each other. Thus, the computation cost of $\pi_{\bar{U}}$ is parallelizable. Also, the computation cost of multiplying two random group elements can be ignored as it is negligible compared to random group exponentiations. See Table 2.

Table 2: Computation cost of voter and bulletin board to verify votes, where $\mathbb{E}_x$ =group exp., $\mathbb{P}_e$ =polynomial evaluation, $\mathcal{M}_{\mathbb{G}}$ = Multiplication in group $\mathbb{G}$

ZKP	**Voter computation**	m **Proofs Verification**
Λ_{RO}^{indv}	$2(n+\ell)[\mathbb{E}_x + \mathbb{P}_x]$	$m(n+\ell)[2\mathbb{E}_x + \mathbb{P}_e + \mathcal{M}_{\mathbb{G}}]$
$\pi_{\bar{U}}$	$6\ell\mathbb{E}_x$	$8\ell m\mathbb{E}_x$

5 A Randomness Beacon alternative to ALBATROSS

Randomness Beacon is required in applications like e-voting and anonymous messaging to provide fresh unbiased random values to all the parties. In 2020,

Cascudo and David published ALBATROSS [9], the state-of-the art randomness beacon protocol based on Packed PVSS. In ALBATROSS each party reveals their secrets as part of the randomness beacon protocol, but to prove that the revealed secrets are valid they have to reveal the whole secret polynomial itself. Thus, if someone wants to verify the secrets' validity then they have to simulate the whole sharing phase of the underlying PVSS protocol, this is a costly computation procedure incurring a cost of quadratic in the number of parties.

In this Section, we present our randomness beacon protocol in Figures 8 and 9. We also show that in many cases our construction is efficient than ALBATROSS. To put simply, we replace the building block being Packed PVSS with our 3PVSS Λ_{RO}^{single}, which additionally gives a single commitment that represents all secret random values of the dealer. In Subsection 5.1, we discuss a bottleneck in ALBATROSS protocol, followed by our alternative construction for ALBATROSS. By the end of this section, we not only show that our protocol outperforms ALBATROSS in many cases but also acknowledge the case where we are not computationally efficient. The main highlight we show is that in terms of dealer communication, we outperform ALBATROSS in every case.

5.1 On the Bottleneck of ALBATROSS

The main bottleneck of ALBATROSS is in its *Reveal* phase (description in Figure 8 in Section 4.3 of [9]). More precisely, the goal of the *Reveal* phase is to reveal the secrets of whom the dealer has done the secret sharing. But in order to verify that those values are actual secrets, in ALBATROSS, a verifier has to simulate the whole sharing phase of the dealer with the values that were revealed. This is because, the underlying proof system of a general PVSS says that *if the PVSS verification algorithm outputs true, then the encryptions of the secret shares correspond to some unique secret with high probability.*

Whereas, in the case of (P)PPVSS, *if the verification algorithm outputs true then it means that the encryptions of the secret shares correspond to the unique secret(s) which was already committed by the dealer with high probability.* So, the intuition is that if one uses a Pre-Constructed PVSS, that can commit to multiple secrets in a single polynomial, over the standard PVSS used in ALBATROSS (a Random Beacon protocol) then we show that the efficiency can be improved.

5.2 Security analysis

The major changes in our protocol in contrast to ALBATROSS are the *Commit* and *Reveal* phases. As long as the randomizers used to commit (/secret keys to decrypt) secrets are not leaked then *Commit* phase is as secure such that it does not reveal anything about the secrets. This is because of the mathematical security guarantees achieved through the sharing phase of our 3PVSS, Λ_{RO}^{single}.

In ALBATROSS, the whole point of the *Reveal* phase is to open the secrets itself, but to verify that the secrets are valid for a given secret shares, one has to simulate the whole sharing phase of the respective dealer. As our 3PVSS, Λ_{RO}^{single}, already proves that the commitment is a valid commitment of dealers'

Randomness Beacon using 3PVSS, Λ_{RO}^{single}

Our protocol with 3PVSS is run between a set $\mathcal{P}$ of n parties $P_1, \ldots, P_n$ who have access to a public ledger where they can post information for later verification. It is assumed that the Setup phase of Λ_{RO}^{single} is already done and the public keys pk_i of each party P_i along with $\{\mathbb{P}_i\}_{i=1}^{\ell}$ being Commitment keys (or public keys of target people) to encrypt the ℓ secrets are already registered in the ledger. In addition, the parties have agreed on a Vandermonde $(n-2t) \times (n-t)$-matrix $M = M(\omega, n-2t, n-t)$ with $\omega \in \mathbb{Z}_q^*$.

1. **Commit:** For $1 \leq j \leq n$:
   - Shareholder P_j executes the Distribution phase of the PPPVSS as Dealer for $\ell = n - 2t$ secrets, publishing commitments (/encryptions) of secrets, $y^j_{-(\ell-1)}, \ldots, y^j_{-1}, y^j_0$, and encryptions of shares $\{y^j_i\}_{i=1}^n$ along with π^j_{proof}, which is a NIZK AoK for proving the correctness of committed(/encrypted) secrets and encrypted secret shares on the public ledger, also learning the secrets $h^{s^j_0}, ..., h^{s^j_{-(\ell-1)}}$ and their corresponding exponents $s^j_0, \ldots, s^j_{-(\ell-1)}$.
2. **Reveal:**
   - Each shareholder checks the validity of the proof π^j_{proof}, i.e., the *verification phase of 3PVSS protocol* Λ_{RO}^{single}.
   - After a set $\mathcal{C}$ containing at least $n-t$ shareholders publish their shares in the public ledger, $P_j \in \mathcal{C}$ reveals ℓ secrets.
   - Every party verifies the validity of secrets by reproducing the commitments using the commitment keys (/public keys of target people).
   - At this point, if every party in $\mathcal{C}$ has opened their secrets correctly, go to step 4' in Figure 9. Otherwise, proceed to step 3 in Figure 9.

Fig. 8: Commit and Reveal phase of the Randomness Beacon using 3PVSS

secrets and their respective secret sharing is done correctly, it does not require to perform the whole simulation of the sharing phase of any dealer. Because the dealer has already committed to the secrets, he cannot cheat by revealing wrong secrets. Therefore, the *Reveal* phase of our protocol also is secure due to the security guarantees of Λ_{RO}^{single}.

5.3 Computational Cost analysis

The dealer has to do a bit more work in the case of our protocol in contrast to ALBATROSS, more explicitly, they have to compute ℓ more group exponentiations and polynomial evaluations. Moreover as a consequence, we decrease computational cost in the *Reveal* phase whenever $\ell < \frac{n(n-t-1)}{2(n-1)}$, roughly speaking, if the number of secrets are less than half of the honest parties then we always perform better in terms of computation when compared to the ALBATROSS.

More precisely, in the reveal phase, we observe that our protocol gains at least 30% more efficiency for performing polynomial evaluations, and at least

Randomness Beacon using 3PVSS, Λ_{RO}^{single} (cont.)

3. **Recovery:** Let $\mathcal{C}_a$ be the set containing at most t malicious shareholders(as Dealers) who did not open the exponents corresponding to their ℓ secrets, $\{h^{s_i^k}\}_{i=0}^{-(\ell-1)}$ for each $P_k \in \mathcal{C}_a$, in *Reveal* phase.
   - Every shareholder P_j should decrypt the secret share of each malicious shareholder(Dealer) in $\mathcal{C}_a$, and give a DLEQ proof (reviewed in Appendix) which asserts that the decryption is performed correctly,i.e., each shareholder should perform the *pessimistic* reconstruction phase of the 3PVSS Λ_{RO}^{single} for every shareholder(Dealer) who has not revealed the exponents corresponding to their secrets.

4 **Output:** Let T be the $(n-t)\times\ell$ matrix with rows indexed by the shareholders in $\mathcal{C}$ and where the row corresponding to $P_a \in \mathcal{C}$ is $(h^{s_0^a}, ..., h^{s_{-(\ell-1)}^a})$.
   - Each computes the $\ell \times \ell$-matrix $R = M \circ T$ by applying FFTE to each column $T^{(j)}$ of T, resulting in column $R^{(j)}$ of R (since $R^{(j)} = M \circ T^{(j)}$ and M is Vandermonde) for $j \in [0, \ell-1]$.
   - Shareholders output the ℓ^2 elements of R as final randomness.

4' **Alternative Output:** if every party in $\mathcal{C}$ has opened her secrets correctly in step *Reveal*, then:
   - Shareholders compute $R = M \circ T$ in the following way:
     Let S be the $(n-t)\times\ell$ matrix with rows indexed by the shareholders in $\mathcal{C}$ and where the row corresponding to $P_a \in \mathcal{C}$ is $(s_0^a, ..., s_{-(\ell-1)}^a)$. Then each party computes $U = M \circ S \in \mathbb{Z}_q^{\ell\times\ell}$ (using the standard FFT in $\mathbb{Z}_q$ to compute each column) and $R = h^U$.
   - Shareholders output the ℓ^2 elements of R as final randomness.

Fig. 9: Recovery and Output phase of the Randomness Beacon using 3PVSS

23% more efficiency for performing group exponentiations when compared to ALBATROSS in the robust case where the number of secrets ℓ is equal to one. See plots 10a and 10b for the performance analysis of polynomial evaluations and group exponentiations in the reveal phase of our protocol and ALBATROSS.

5.4 Communication Cost analysis

Though the dealer has to communicate an extra group element compared to ALBATROSS in *commit phase*, as a consequence, for a fixed number of secrets, the dealers' communication cost is constant in *reveal* phase as opposed to linear in number of corrupted parties in ALBATROSS.

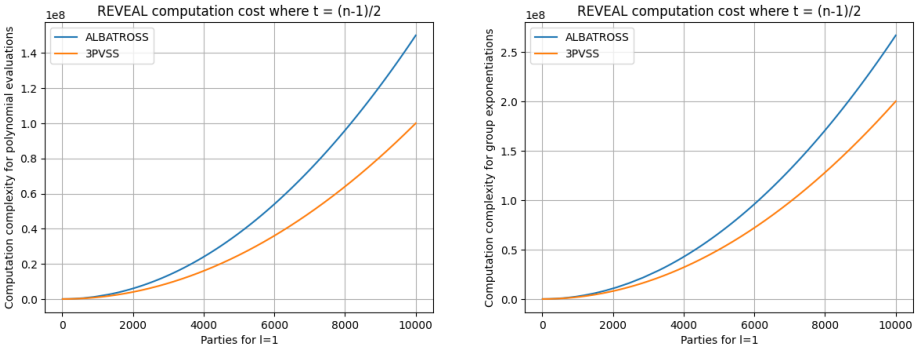
In the plots 11b and 11a, we show the communication cost of a dealer to bare in ALBATROSS and our protocol in both the commit and reveal phases.

Table 3: Computational cost of dealer and shareholders, $\mathbb{E}_x$ =group exponentiation and $\mathbb{P}_e$ =polynomial evaluation in group $\mathbb{G}$ of order q, for q a large prime

Protocol	**Output size**	**Commit** *(by Dealer)*	**Reveal** *(by shareholder)*	**Recovery** *(by shareholder)*
ALBATROSS *Honest case*	ℓ^2	$(2n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	**Share Verification -** $(n-1)n[2\mathbb{E}_x+\mathbb{P}_e]$ **Secret Verification -** $(n-1)(n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	-
3PVSS *Honest case*	ℓ^2	$2(n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	**Share Verification -** $(n-1)(n+\ell)[2\mathbb{E}_x+\mathbb{P}_e]$ **Secret Verification -** $(n-1)\ell\mathbb{E}_x$	-
ALBATROSS *Robust case*	ℓ^2	$(2n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	**Share Verification -** $(n-1)n[2\mathbb{E}_x+\mathbb{P}_e]$ **Secret Verification -** $(n-t-1)(n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	$[3+4(n-t)]t\mathbb{E}_x$
3PVSS *Robust case*	ℓ^2	$2(n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	**Share Verification -** $(n-1)(n+\ell)[2\mathbb{E}_x+\mathbb{P}_e]$ **Secret Verification -** $(n-t-1)\ell\mathbb{E}_x$	$[3+4(n-t)]t\mathbb{E}_x$

Table 4: Communication cost of dealer and (each) shareholder, $\mathbb{G}$ =group of order q and $\mathbb{Z}_q$ = modular group of order q, where q is a large prime

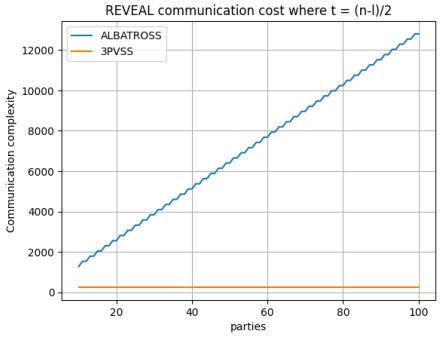
Protocol	**Commit** *(by Dealer)*	**Reveal** *(by Dealer)*	**Recovery** *(by shareholder)*
ALBATROSS	$n\mathbb{G}+(t+\ell)\mathbb{Z}_q$	$(t+\ell)\mathbb{Z}_q$	$1\mathbb{G}+2\mathbb{Z}_q$
with PPPVSS	$(n+1)\mathbb{G}+(t+\ell)\mathbb{Z}_q$	$\ell\mathbb{Z}_q$	$1\mathbb{G}+2\mathbb{Z}_q$



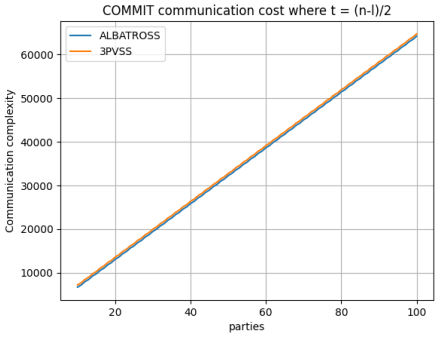
(a) Polynomial evaluations cost in *reveal phase*

(b) Group exponentiations cost in *reveal phase*

Fig. 10: Computation costs bore by each verifier in the *reveal phase* for the most robust case where $\ell = 1$



(a) Communication cost of a dealer in reveal phase of ALBATROSS and our protocol



(b) Communication cost of a dealer in commit phase phase of ALBATROSS and our protocol

Fig. 11: Comparing *communication costs* of each shareholder as the dealer in *commit* and *reveal* phases

6 Conclusion

We introduced *Packed Pre-constructed Publicly Verifiable Secret Sharing* (3PVSS), a new primitive that combines the benefits of packed secret sharing with the pre-constructability property of PPVSS. Our construction enables a dealer to distribute multiple secrets simultaneously while committing to the reconstructed values, thereby preserving the optimistic reconstruction approach and public verifiability that make PPVSS attractive for practical distributed protocols.

We presented two practical 3PVSS constructions with complementary properties. The first construction, Λ_{RO}^{indv}, publishes individual commitments to each secret, offering greater flexibility and enabling applications that require independent verification of shared values. The second construction, Λ_{RO}^{single}, publishes a single commitment to all secrets, resulting in improved communication efficiency. Together, these constructions demonstrate that the 3PVSS paradigm can support different design goals depending on the requirements of the application.

To illustrate the usefulness of our approach, we applied the two constructions to two important distributed protocols. Using Λ_{RO}^{single}, we revisited the ALBATROSS randomness beacon protocol and obtained a more efficient variant with reduced computational and communication costs. Using Λ_{RO}^{indv}, we extended a recently proposed universally verifiable e-voting protocol to support multi-candidate elections while maintaining efficiency and public verifiability.

Overall, our results show that extending PPVSS to the packed setting provides a powerful tool for designing more efficient distributed cryptographic protocols. We believe that the 3PVSS paradigm can serve as a useful building block in a variety of applications beyond those considered in this work, including distributed key generation, threshold cryptography, and secure multi-party computation. Exploring further applications of 3PVSS and extending our constructions to other PPVSS frameworks remain interesting directions for future research.

Acknowledgments

We would like to thank Mahdi Rahimi for helpful discussions. This work was supported by the Flemish Government through the Cybersecurity Research Program with grant number VOEWICS02.

References

1. Shahla Atapoor, Karim Baghery, Daniele Cozzo, and Robi Pedersen. VSS from distributed ZK proofs and applications. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part I*, volume 14438 of *Lecture Notes in Computer Science*, pages 405–440, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
2. Shahla Atapoor, Karim Baghery, Georgio Nicolas, Robi Pedersen, and Jannik Spiessens. Batched and packed (publicly) verifiable secret sharing: A unified framework and applications. Cryptology ePrint Archive, Report 2025/2018, 2025.
3. Michael Backes, Aniket Kate, and Arpita Patra. Computational verifiable secret sharing revisited. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 590–609, Seoul, South Korea, December 4–8, 2011. Springer Berlin Heidelberg, Germany.
4. Karim Baghery. Π: A unified framework for computational verifiable secret sharing. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025: 28th International Conference on Theory and Practice of Public Key Cryptography, Part IV*, volume 15677 of *Lecture Notes in Computer Science*, pages 110–142, Røros, Norway, May 12–15, 2025. Springer, Cham, Switzerland.
5. Karim Baghery, Noah Knapen, Georgio Nicolas, and Mahdi Rahimi. Preconstructed publicly verifiable secret sharing and applications. In Marc Fischlin and Veelasha Moonsamy, editors, *ACNS 2025: 23rd International Conference on Applied Cryptography and Network Security, Part I*, volume 15825 of *Lecture Notes in Computer Science*, pages 89–119, Munich, Germany, June 23–26, 2025. Springer, Cham, Switzerland.
6. George Robert Blakley. Safeguarding Cryptographic Keys. In *1979 International workshop on managing requirements knowledge*, pages 313–318. IEEE Computer Society, 1979.
7. Ignacio Cascudo, Daniele Cozzo, and Emanuele Giunta. Verifiable secret sharing from symmetric key cryptography with improved optimistic complexity. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part VII*, volume 15490 of *Lecture Notes in Computer Science*, pages 100–128, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
8. Ignacio Cascudo and Bernardo David. SCRAPE: Scalable randomness attested by public entities. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 2017: 15th International Conference on Applied Cryptography and Network Security*, volume 10355 of *Lecture Notes in Computer Science*, pages 537–556, Kanazawa, Japan, July 10–12, 2017. Springer, Cham, Switzerland.
9. Ignacio Cascudo and Bernardo David. ALBATROSS: Publicly AttestabLe BATched Randomness based On Secret Sharing. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 311–341, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.

10. Ignacio Cascudo and Bernardo David. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part V*, volume 14655 of *Lecture Notes in Computer Science*, pages 216–248, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
11. Ignacio Cascudo, Bernardo David, Lydia Garms, and Anders Konring. YOLO YOSO: Fast and simple encryption and secret sharing in the YOSO model. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 651–680, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.
12. David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO'92*, volume 740 of *Lecture Notes in Computer Science*, pages 89–105, Santa Barbara, CA, USA, August 16–20, 1993. Springer Berlin Heidelberg, Germany.
13. Benny Chor, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. Verifiable Secret Sharing and Achieving Simultaneity in the Presence of Faults. In *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, pages 383–395. IEEE, 1985.
14. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In Yvo Desmedt, editor, *Advances in Cryptology – CRYPTO'94*, volume 839 of *Lecture Notes in Computer Science*, pages 174–187, Santa Barbara, CA, USA, August 21–25, 1994. Springer Berlin Heidelberg, Germany.
15. Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science*, pages 427–437, Los Angeles, CA, USA, October 12–14, 1987. IEEE Computer Society Press.
16. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
17. Matthew K. Franklin and Moti Yung. Communication complexity of secure computation (extended abstract). In *24th Annual ACM Symposium on Theory of Computing*, pages 699–710, Victoria, BC, Canada, May 4–6, 1992. ACM Press.
18. Craig Gentry, Shai Halevi, and Vadim Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 458–487, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
19. Jens Groth. Non-interactive distributed key generation and key resharing. Cryptology ePrint Archive, Report 2021/339, 2021.
20. Somayeh Heidarvand and Jorge L Villar. Public Verifiability from Pairings in Secret Sharing Schemes. In *International Workshop on Selected Areas in Cryptography*, pages 294–308. Springer, 2008.
21. Aniket Kate, Easwar Vivek Mangipudi, Pratyay Mukherjee, Hamza Saleem, and Sri Aravinda Krishnan Thyagarajan. Non-interactive VSS using class groups and application to DKG. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 4286–4300, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.
22. Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO'91*,

volume 576 of *Lecture Notes in Computer Science*, pages 129–140, Santa Barbara, CA, USA, August 11–15, 1992. Springer Berlin Heidelberg, Germany.
23. Berry Schoenmakers. A simple publicly verifiable secret sharing scheme and its application to electronic. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO'99*, volume 1666 of *Lecture Notes in Computer Science*, pages 148–164, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.
24. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
25. Avi Wigderson, MB Or, and S Goldwasser. Completeness Theorems for Noncryptographic Fault-Tolerant Distributed Computations. In *Proceedings of the 20th Annual Symposium on the Theory of Computing (STOC88)*, pages 1–10, 1988.

Appendix:

A On Hardness of modified-PDL Problem

In this section, we reduce the security of modified-PDL (given in Def. 3) to the standard DL assumption. We present the reduction for the representative case $t = 1$, $\ell = 2$, and $n \geq 4$, where the hidden polynomial has degree at most 2. The reduction embeds the DL challenge as one polynomial evaluation and then simulates the entire modified-PDL transcript for the adversary. In particular, the remaining group elements, including the aggregate term F_0, are generated by choosing sufficiently many polynomial evaluations at random and interpolating the rest in the exponent.

Theorem 4 (DL-hardness of modified-PDL, case $t = 1, \ell = 2$). *Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of prime order q. Suppose there exists a probabilistic polynomial-time algorithm $\mathcal{A}$ that solves the modified-PDL problem for parameters $t = 1, \ell = 2$ and $n \geq 4$ with probability ϵ. Then there exists a probabilistic polynomial-time algorithm $\mathcal{B}$ that solves the discrete logarithm problem in $\mathbb{G}$ with the same probability ϵ, up to the negligible probability of sampling collisions.*

Proof. Let (g, h), where $h = g^a$ for a uniformly random unknown $a \in \mathbb{Z}_q$, be a DL challenge. We construct an algorithm $\mathcal{B}$ that uses $\mathcal{A}$ to recover a.

For $t = 1$ and $\ell = 2$, the modified-PDL polynomial has degree at most $t + \ell - 1 = 2$. Algorithm $\mathcal{B}$ first samples distinct points

$$x_{-1}, x_0, x_1, \ldots, x_n \in \mathbb{Z}_q,$$

with $n \geq 4$. It also samples independently and uniformly random nonzero exponents

$$\alpha_{-1}, \alpha_0 \in \mathbb{Z}_q^\star$$

such that $g_{-1} := g^{\alpha_{-1}}$ and $g_0 := g^{\alpha_0}$ are distinct from each other and from g. This only fails with negligible probability, and conditioned on no collision, g_{-1} and g_0 are distributed as random distinct generators of $\mathbb{G}$.

Next, $\mathcal{B}$ embeds the unknown discrete logarithm a as the value of the hidden polynomial at x_1. To this end, it samples random values

$$y_2, y_3 \xleftarrow{\$} \mathbb{Z}_q$$

and implicitly defines the unique degree-at-most-two polynomial $f \in \mathbb{Z}_q[X]$ satisfying

$$f(x_1) = a, \qquad f(x_2) = y_2, \qquad f(x_3) = y_3.$$

Although $\mathcal{B}$ does not know a, it can compute all required group elements by interpolation in the exponent. Let $\lambda_1(X), \lambda_2(X), \lambda_3(X)$ be the Lagrange basis polynomials associated with the points x_1, x_2, x_3, namely

$$\lambda_i(X) = \prod_{\substack{k \in \{1,2,3\} \\ k \neq i}} \frac{X - x_k}{x_i - x_k}.$$

Then for every $z \in \mathbb{Z}_q$,

$$f(z) = a\lambda_1(z) + y_2\lambda_2(z) + y_3\lambda_3(z).$$

Therefore, for every $i \in \{1, \ldots, n\}$, algorithm $\mathcal{B}$ can set

$$F_i := h^{\lambda_1(x_i)} g^{y_2\lambda_2(x_i) + y_3\lambda_3(x_i)}.$$

Indeed,

$$F_i = (g^a)^{\lambda_1(x_i)}\, g^{y_2\lambda_2(x_i) + y_3\lambda_3(x_i)} = g^{f(x_i)}.$$

In particular, $F_1 = h = g^a$, $F_2 = g^{y_2}$, and $F_3 = g^{y_3}$. It remains to simulate the modified commitment term

$$F_0 = g_{-1}^{f(x_{-1})} g_0^{f(x_0)}.$$

Using $g_{-1} = g^{\alpha_{-1}}$ and $g_0 = g^{\alpha_0}$, we have

$$F_0 = g^{\alpha_{-1} f(x_{-1}) + \alpha_0 f(x_0)}.$$

By interpolation,

$$f(x_{-1}) = a\lambda_1(x_{-1}) + y_2\lambda_2(x_{-1}) + y_3\lambda_3(x_{-1}),$$

and similarly

$$f(x_0) = a\lambda_1(x_0) + y_2\lambda_2(x_0) + y_3\lambda_3(x_0).$$

Hence

$$\begin{aligned} F_0 &= g^{a(\alpha_{-1}\lambda_1(x_{-1}) + \alpha_0\lambda_1(x_0)) + y_2(\alpha_{-1}\lambda_2(x_{-1}) + \alpha_0\lambda_2(x_0)) + y_3(\alpha_{-1}\lambda_3(x_{-1}) + \alpha_0\lambda_3(x_0))} \\ &= h^{\alpha_{-1}\lambda_1(x_{-1}) + \alpha_0\lambda_1(x_0)} g^{y_2(\alpha_{-1}\lambda_2(x_{-1}) + \alpha_0\lambda_2(x_0)) + y_3(\alpha_{-1}\lambda_3(x_{-1}) + \alpha_0\lambda_3(x_0))}. \end{aligned}$$

Thus $\mathcal{B}$ can compute F_0 without knowing a. Next, algorithm $\mathcal{B}$ gives $\mathcal{A}$ the modified-PDL instance

$$(x_{-1}, x_0, x_1, \ldots, x_n, g, g_{-1}, g_0, F_0, F_1, \ldots, F_n)\,.$$

By construction, this instance is distributed exactly as a genuine modified-PDL instance for a random degree-at-most-two polynomial f, except with negligible statistical distance due to the rejection of generator collisions.

Suppose $\mathcal{A}$ succeeds and outputs a polynomial $\hat{f} \in \mathbb{Z}_q[X]$ of degree at most 2 satisfying

$$F_0 = g_{-1}^{\hat{f}(x_{-1})} g_0^{\hat{f}(x_0)} \qquad \text{and} \qquad F_i = g^{\hat{f}(x_i)} \quad \text{for } 1 \le i \le n.$$

Since $F_1 = h = g^a$, we obtain

$$g^{\hat{f}(x_1)} = F_1 = h = g^a.$$

As $\mathbb{G}$ has prime order q, this implies

$$\hat{f}(x_1) = a \pmod{q}.$$

Therefore, $\mathcal{B}$ outputs

$$a := \hat{f}(x_1).$$

Thus, whenever $\mathcal{A}$ solves the modified-PDL instance, $\mathcal{B}$ solves the original DL challenge. Consequently,

$$\mathsf{Adv}^{\mathcal{B}}_{\mathsf{DL}} \ge \mathsf{Adv}^{\mathcal{A}}_{\mathsf{mPDL}} - negl(\lambda),$$

where the negligible term accounts only for the probability of sampling collisions among the generators or evaluation points. This proves that modified-PDL is at least as hard as DL for $t = 1, \ell = 2$ and $n \ge 4$. □

B Efficiency Comparisons

B.1 Comparing Costs of 3PVSS Schemes

In this Subsection, we analyze the computational and communication cost of both our 3PVSS schemes Λ^{indv}_{RO} and Λ^{single}_{RO}. One can explicitly find the Table 5 which summarizes these costs.

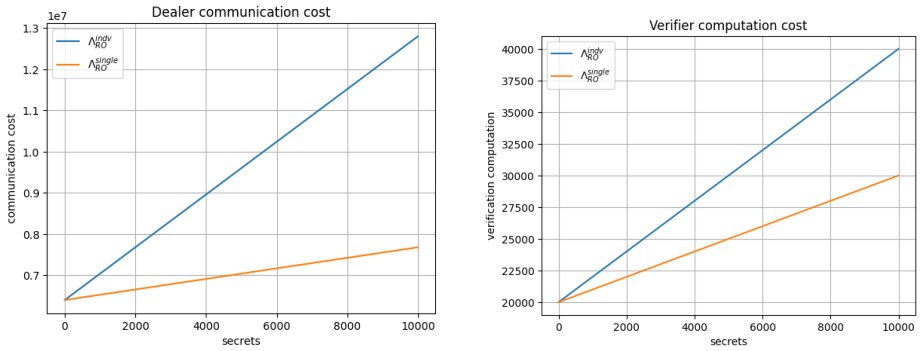
In Figures 12a and 12b, one can find heuristics for dealer communication costs and proof verification costs of π_{3PVSS} for fixed number of shareholders n but varying corruption threshold t and the number of secrets ℓ.

On the communication cost Apart from downloading the public values from the initialization phase, the communication cost of Λ^{single}_{RO} is independent of the number of secrets in contrast to Λ^{indv}_{RO}, where the communication cost grows linearly with the number of secrets. This is the main advantage of using Λ^{single}_{RO} over Λ^{indv}_{RO}.

In the graph, we fixed the number of parties, $n = 10000$, for which we varied ℓ with $1 \le t \le \frac{n-\ell}{2}$.

Table 5: Cost comparisons between Λ_{RO}^{indv} and Λ_{RO}^{single}. BC: Dealer's Broadcast size, Dow: Download size by the verifier, Opt. Recon: Optimistic Reconstruction, Pes. Recon: Pessimistic Reconstruction. $\mathbb{E}_x$: Group exponentiation, $\mathbb{P}_e$: Polynomial evaluation, $M_{\mathbb{G}}$: multiplication in the cyclic group $\mathbb{G}$ of prime order q, $\mathbb{Z}_q$: modular prime group under addition.

Scheme	Share	Broadcast, Dow.	Verification	Opt. Recon	Pes. Recon
Λ_{RO}^{indv}	$2(n+\ell)[\mathbb{E}_x+\mathbb{P}_e]$	**BC:** $(n+\ell)\lvert\mathbb{G}\rvert+(t+\ell+1)\lvert\mathbb{Z}_p\rvert$ **Dow:** $(2n+\ell+1)\lvert\mathbb{G}\rvert+(t+\ell+1)\lvert\mathbb{Z}_p\rvert$	$(n+\ell)[2\mathbb{E}_x+\mathbb{P}_e+M_{\mathbb{G}}]$	$\ell\mathbb{E}_x$	$5(t+\ell)\mathbb{E}_x+t\ell M_{\mathbb{G}}$
Λ_{RO}^{single}	$2(n+\ell)[\mathbb{E}_x+\mathbb{P}_e]+2\ell M_{\mathbb{G}}$	**BC:** $(n+1)\lvert\mathbb{G}\rvert+(t+\ell+1)\lvert\mathbb{Z}_p\rvert$ **Dow:** $(2n+\ell+1)\lvert\mathbb{G}\rvert+(t+\ell+1)\lvert\mathbb{Z}_p\rvert$	$(2n+\ell+1)\mathbb{E}_x+(n+\ell)\mathbb{P}_e+(n+\ell+1)M_{\mathbb{G}}$	$\ell(\mathbb{E}_x+M_{\mathbb{G}})$	$5(t+\ell)\mathbb{E}_x+t\ell M_{\mathbb{G}}$



(a) Dealer communication cost in y-axis for varying ℓ and threshold t

(b) Verifier computation cost in y-axis for varying ℓ and threshold t

Fig. 12: The x-axis represents the possible number of secrets ℓ when the total number of parties n is equal to 10000. The considered group is a cyclic elliptic curve over 256-bit prime field, that has order a 256-bit prime q

On the computational cost As Λ_{RO}^{indv} is a direct extension of Λ_{RO} [5], the computational cost grows linearly with the number of secrets. And as Λ_{RO}^{single} is a slightly different approach, we save number of exponentiations in verification by a constant factor of ℓ but it costs additional group multiplications in share, verification and optimistic reconstruction phases. As group exponentiations with random exponents are usually more expensive than group multiplications (to put that in the perspective, computing g^x for a random $x \in \mathbb{Z}_q$ can take $\mathcal{O}(\log q)$ group multiplications). Hence, the realistic increase in cost of Λ_{RO}^{single} due to additional group multiplications can be negligible.

B.2 Randomness beacon detailed cost analysis

Computational Complexity. One can notice that the major changes are in the commit and reveal phase of the protocol. Hence, in this Subsection we only talk about the performance analysis in those phases. See Table 3 for an overview. Also, we ignore the number of multiplications in the group $\mathbb{G}$, i.e., $M_{\mathbb{G}}$, as it is negligible compared to the number of group exponentiations $\mathbb{E}_x$ with random exponents in practice.

- In ALBATROSS, a dealer(as a part of **commit**) performs $n(\mathbb{E}_x + \mathbb{P}_e)$ to compute commitments and to give a proof, they require to additionally perform $n(\mathbb{E}_x + \mathbb{P}_e)$. Also, the dealer performs $\ell(\mathbb{E}_x + \mathbb{P}_e)$ for computing secrets and keeping it to himself. In total dealer needs to do $(2n + \ell)[\mathbb{E}_x + \mathbb{P}_e]$.
  - In **Reveal**, a verifier should compute $2n\mathbb{E}_x$ which internally requires additional $n\mathbb{P}_e$, i.e., in total it requires $(n-1)n(2\mathbb{E}_x + \mathbb{P}_e)$ computations for each verifier.
    - In **Robust case** where t dealers do not open their polynomials, a verifier requires to verify $n-t$ polynomials of honest dealers, i.e., for each honest dealer, a verifier performs $n\mathbb{P}_e$ to evaluate secret share exponents and does $n\mathbb{E}_x$ to get secret shares and cross checks them in the public ledger. Also, finally the verifier computes $\ell\mathbb{P}_e$ to get secret exponents and get ℓ secrets by doing $\ell\mathbb{E}_x$. As there are $n-t$ honest dealers, the verifier has to compute $(n-t)(n+\ell)(\mathbb{E}_x + \mathbb{P}_e)$.
    - In **Honest case**, As everyone is honest, each verifier does $(n-1)(n+\ell)(\mathbb{E}_x + \mathbb{P}_e)$.
  - **Recovery** phase only exists if some party does not open the polynomial leading to PVSS reconstruction phase, in the worst case there should be reconstruction for the secrets of t malicious parties. Given a malicious shareholder who has not opened the secret polynomial, each shareholder/re-constructor has to decrypt their share, which requires $1\mathbb{E}_x$ and should give a DLEQ proof that they have decrypted correctly, which additionally requires $2\mathbb{E}_x$; Also the re-constructor should verify DLEQ proofs of correct share decryption from $n-t$ honest shareholders requiring them to do $4(n-t)\mathbb{E}_x$. In total, each re-constructor requires $[3 + 4(n-t)]t\mathbb{E}_x$.
- Using 3PVSS in randomness beacon protocol, a dealer(as a part of **commit**) requires to do $(n+\ell)[\mathbb{E}_x + \mathbb{P}_e]$ and $(\ell-1)M_{\mathbb{G}}$ to compute $\{y_i\}_{i=0}^{n}$. For generating the proof that y_i's are valid encryptions of the secret shares and also y_0 is a commitment of the ℓ secrets, the dealer should do $(n+\ell)[\mathbb{E}_x + \mathbb{P}_e]$ which internally requires additional $(\ell-1)M_{\mathbb{G}}$. In total, a dealer has to do $2\left[(n+\ell)[\mathbb{E}_x + \mathbb{P}_e] + (\ell-1)M_{\mathbb{G}}\right]$.
  - In **Reveal**, a verifier should do $(n+\ell)(2\mathbb{E}_x + \mathbb{P}_e)$ and $(\ell-1)M_{\mathbb{G}}$ for each proof. In total, a verifier has to do $(n-1)(n+\ell)[2\mathbb{E}_x + \mathbb{P}_e] + (n-1)(\ell-1)M_{\mathbb{G}}$.
    - In **Robust case** with t malicious parties not opening the secret polynomials, a verifier should do $\ell\mathbb{E}_x + (\ell-1)M_{\mathbb{G}}$ to verify each proof, so in total each verifier should do $(n-t-1)[l\mathbb{E}_x + (\ell-1)M_{\mathbb{G}}]$.

  - In **Honest case** where everyone is honest, a verifier will do $(n-1)\ell(\mathbb{E}_x + M_{\mathbb{G}})$.
- The computational complexity of each re-constructor in **Recovery** phase is exactly same as in the case of ALBATROSS.

Communication Complexity. See Table 4 for an overview.

- In ALBATROSS, a dealer (as a part of **commit**) should send n group elements as commitments, $t+\ell$ elements in $\mathbb{Z}/q\mathbb{Z}$ that defines the polynomial used in the ZKP and 1 extra element in $\mathbb{Z}/q\mathbb{Z}$ from RO.
  - In **Reveal**, an honest dealer would broadcast $t+\ell$ coefficients in $\mathbb{Z}/q\mathbb{Z}$ concerning the secret polynomial.
  - If some party has not revealed their polynomial, then in **Recovery** phase a re-constructor using PVSS reconstruction protocol should broadcast 1 element in group which is being the decrypted secret, for the proof of correct decryption, they have to broadcast 3 more group elements along with a polynomial which requires $t+\ell$ coefficients in $\mathbb{Z}/q\mathbb{Z}$ and 1 group element from RO.
- Using 3PVSS in randomness beacon protocol, a dealer (as a part of **commit**) should send $n+1$ group elements as commitments, $t+\ell$ elements in $\mathbb{Z}/q\mathbb{Z}$ that defines the polynomial used in the ZKP and 1 extra element in $\mathbb{Z}/q\mathbb{Z}$ from RO.
  - In **Reveal**, an honest dealer would broadcast ℓ elements in $\mathbb{Z}_q$ concerning the exponents to construct the secret.
  - If some part has not revealed their secrets, then the communication cost of each re-constructor is exactly same as in the case of ALBATROSS.